

AWS Assignment

Amazon VPC – Production & Development Networks

XYZ Corporation – step-by-step implementation with screenshots

Item	Detail
AWS Region	US East (N. Virginia) – us-east-1 (Availability Zone us-east-1a)
Production VPC	XYZ-Production-VPC – 10.0.0.0/16 – 4-tier, 5 subnets (1 public, 4 private)
Development VPC	XYZ-Development-VPC – 20.0.0.0/16 – 2-tier, 2 subnets (web, db)
Connectivity	VPC peering XYZ-Prod-Dev-Peering between the two VPCs
Services used	VPC, Subnets, Route Tables, Internet Gateway, NAT Gateway, Network ACLs, Security Groups, EC2, VPC Peering

Contents

1. Problem Statement and Design.....	3
Tasks to be performed.....	3
Architecture overview.....	3
Subnet plan.....	3
Resources created.....	4
2. Production Network (4-Tier Architecture).....	5
Step 1: Create the Production VPC.....	5
Step 2: Create the five subnets.....	5
Step 3: Create and attach the Internet Gateway.....	6
Step 4: Create the NAT Gateway.....	6
Step 5: Create the route tables.....	7
Step 6: Create the security groups.....	11
Step 7: Create the network ACLs.....	12
Step 8: Launch an instance in every subnet.....	13
3. Development Network (2-Tier Architecture).....	15
Step 9: Create the Development VPC.....	15
Step 10: Create the web and db subnets.....	15
Step 11: Enable auto-assign public IP on the web subnet.....	16
Step 12: Create and attach the Internet Gateway.....	16
Step 13: Create the route tables.....	17
Step 14: Create the network ACLs.....	20
Step 15: Create the security groups.....	21
Step 16: Launch the development instances.....	21
4. VPC Peering and DB-to-DB Connection.....	23
Step 17: Create the peering connection.....	23
Step 18: Review the peering details.....	23
Step 19: Add routes for the peering connection.....	24
Step 20: Allow database traffic in the security groups.....	25
5. Final Verification – Resource Maps.....	26
Production VPC resource map.....	26
Development VPC resource map.....	27
6. Conclusion.....	28

1. Problem Statement and Design

XYZ Corporation is expanding and needs a separate Amazon VPC for the production team and for the development team. Each VPC has its own set of requirements, and the two networks must be able to talk to each other at the database layer.

Tasks to be performed

Production Network

1. Design and build a 4-tier architecture.
2. Create 5 subnets: 4 private (app1, app2, dbcache, db) and 1 public (web).
3. Launch instances in all subnets and name them after their subnet.
4. Allow the dbcache instance and the app1 subnet to send internet requests.
5. Manage security groups and NACLs.

Development Network

1. Design and build a 2-tier architecture with two subnets, web and db, and launch an instance in each, named after the subnet.
2. Make sure only the web subnet can send internet requests.
3. Create a peering connection between the production and development networks.
4. Set up a connection between the db subnets of the production and development networks.

Architecture overview

```

PRODUCTION   XYZ-Production-VPC   10.0.0.0/16
Tier 1  web      10.0.1.0/24   public   -> Internet Gateway (+ NAT gateway)
Tier 2  app1     10.0.2.0/24   private  -> NAT (outbound internet)
        app2     10.0.3.0/24   private  (no internet)
Tier 3  dbcache  10.0.4.0/24   private  -> NAT (outbound internet)
Tier 4  db       10.0.5.0/24   private  (no internet)
        ||
        ||   VPC peering: XYZ-Prod-Dev-Peering (db <-> db)
        ||
DEVELOPMENT  XYZ-Development-VPC  20.0.0.0/16
Tier 1  web     20.0.1.0/24   public   -> Internet Gateway
Tier 2  db      20.0.2.0/24   private  (no internet)
  
```

Subnet plan

VPC	Subnet	CIDR	Type	Route table	Internet access
Production	XYZ-prod-web-public	10.0.1.0/24	Public	XYZ-prod-public-RT	In/out via Internet Gateway
Production	XYZ-prod-app1-private	10.0.2.0/24	Private	XYZ-prod-private-internet-RT	Outbound only via NAT
Production	XYZ-prod-app2-private	10.0.3.0/24	Private	XYZ-prod-private-no-internet-RT	None
Production	XYZ-prod-dbcache-private	10.0.4.0/24	Private	XYZ-prod-private-internet-RT	Outbound only via NAT
Production	XYZ-prod-db-private	10.0.5.0/24	Private	XYZ-prod-private-no-internet-RT	None (peering to Dev)

VPC	Subnet	CIDR	Type	Route table	Internet access
Development	XYZ-dev-web	20.0.1.0/24	Public	XYZ-dev-public-RT	In/out via Internet Gateway
Development	XYZ-dev-db	20.0.2.0/24	Private	XYZ-dev-private-RT	None (peering to Prod)

Resources created

Resource	Production	Development
VPC	XYZ-Production-VPC (vpc-072c435f670a4a269)	XYZ-Development-VPC (vpc-080fda18c5d5e1fd3)
Internet gateway	XYZ-prod-IGW (igw-049a7f20d90323418)	XYZ-dev-IGW (igw-0caee2ab47c1117a)
NAT gateway	XYZ-prod-NAT (nat-0b35abda625eb6574), public IP 98.95.167.165	- (not required)
Route tables	XYZ-prod-public-RT, XYZ-prod-private-internet-RT, XYZ-prod-private-no-internet-RT	XYZ-dev-public-RT, XYZ-dev-private-RT
Network ACLs	XYZ-prod-web-NACL, XYZ-prod-app-NACL, XYZ-prod-data-NACL	XYZ-dev-web-NACL, XYZ-dev-db-NACL
Security groups	XYZ-prod-web-SG, XYZ-prod-app-SG, XYZ-prod-dbcache-SG, XYZ-prod-db-SG	XYZ-dev-web-SG, XYZ-dev-db-SG
EC2 instances (t3.micro)	XYZ-prod-web, XYZ-prod-app1, XYZ-prod-app2, XYZ-prod-dbcache, XYZ-prod-db	XYZ-dev-web, XYZ-dev-db
VPC peering	XYZ-Prod-Dev-Peering (pcx-00d8cdf7b2ff72646) – Active	

2. Production Network (4-Tier Architecture)

The production VPC separates the application into four tiers – web, application, cache and database – each in its own subnet with its own route table, network ACL and security group.

Step 1: Create the Production VPC

1. Open the **VPC console** in region **US East (N. Virginia)** and click **Create VPC**.
2. Choose **VPC only**. Name tag: **XYZ-Production-VPC**.
3. IPv4 CIDR block: **10.0.0.0/16**. Tenancy: Default. Click **Create VPC**.

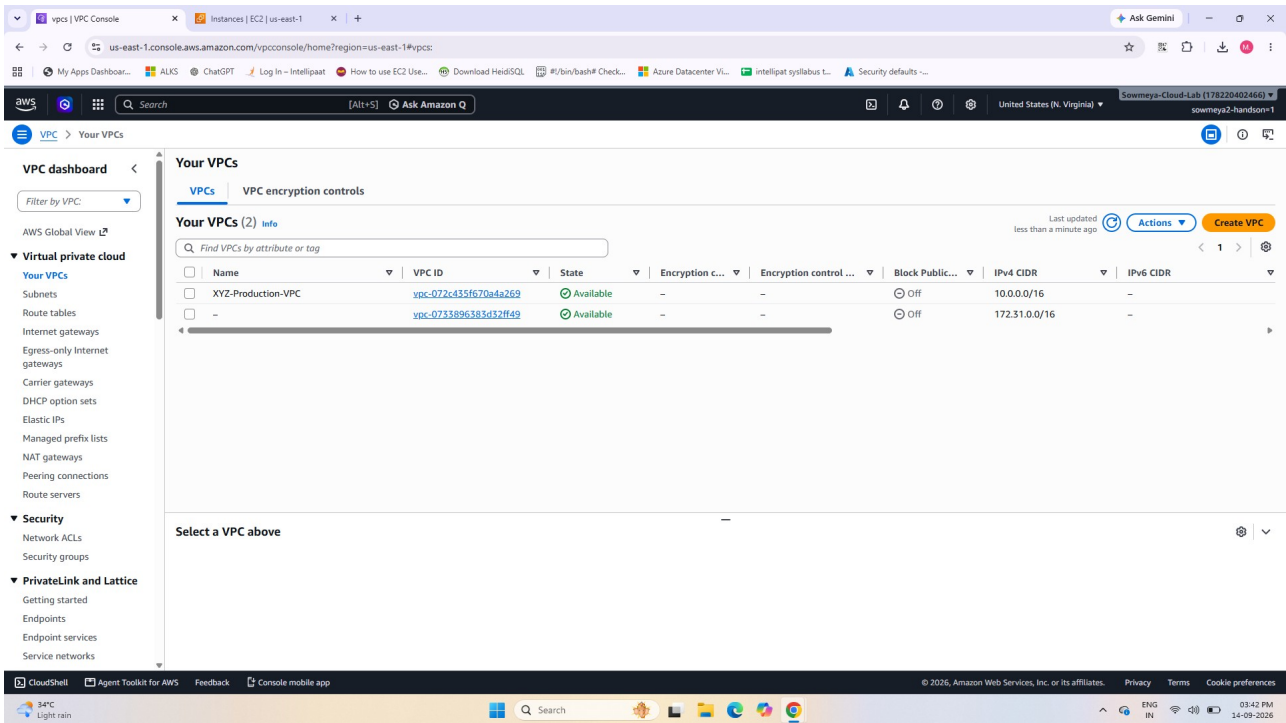


Figure 1: XYZ-Production-VPC created with CIDR 10.0.0.0/16 (state Available)

Step 2: Create the five subnets

1. Go to **Subnets** → **Create subnet** and select **XYZ-Production-VPC**.
2. Add the five subnets below, all in Availability Zone **us-east-1a**:

Subnet name	IPv4 CIDR	Type
XYZ-prod-web-public	10.0.1.0/24	Public (web)
XYZ-prod-app1-private	10.0.2.0/24	Private (app1)
XYZ-prod-app2-private	10.0.3.0/24	Private (app2)
XYZ-prod-dbcache-private	10.0.4.0/24	Private (dbcache)
XYZ-prod-db-private	10.0.5.0/24	Private (db)

1. Select **XYZ-prod-web-public** → **Actions** → **Edit subnet settings** and tick **Enable auto-assign public IPv4 address** (same setting as shown for the development web subnet in Step 11).

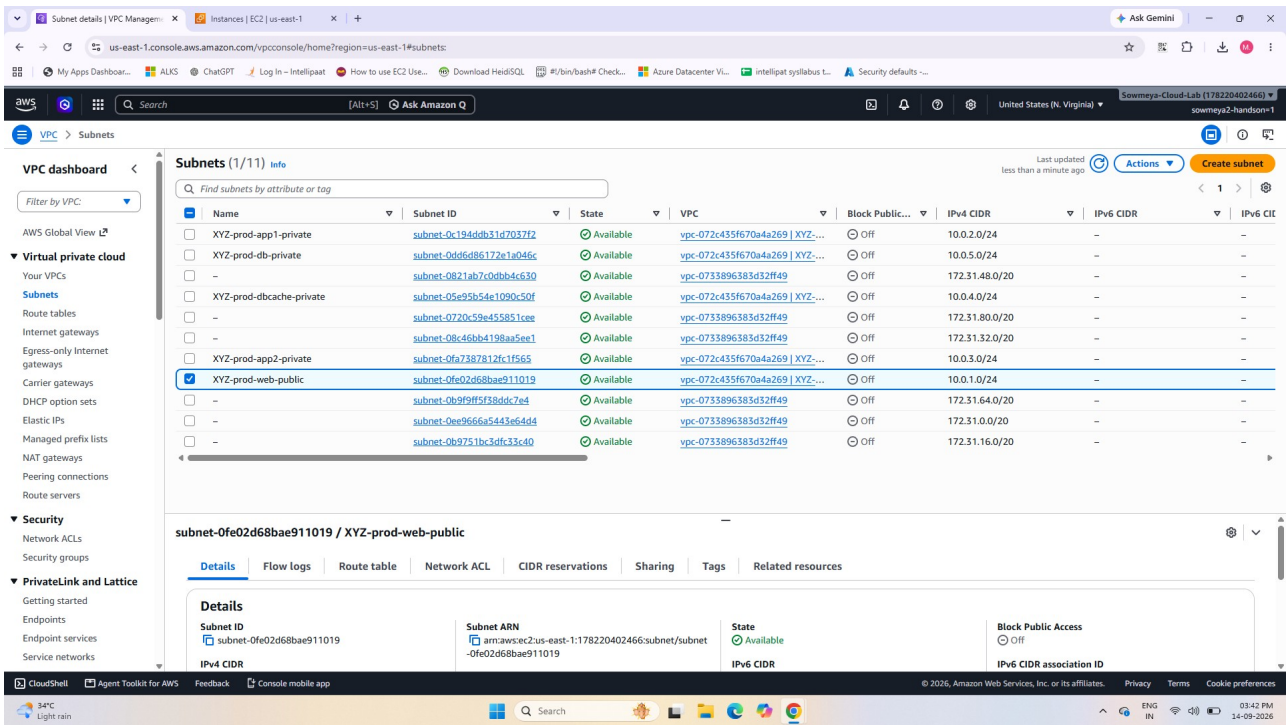


Figure 2: Five production subnets in XYZ-Production-VPC (10.0.1.0/24 – 10.0.5.0/24)

Step 3: Create and attach the Internet Gateway

1. Go to **Internet gateways** → **Create internet gateway**. Name: **XYZ-prod-IGW**.
2. Select it and choose **Actions** → **Attach to VPC** → **XYZ-Production-VPC**.

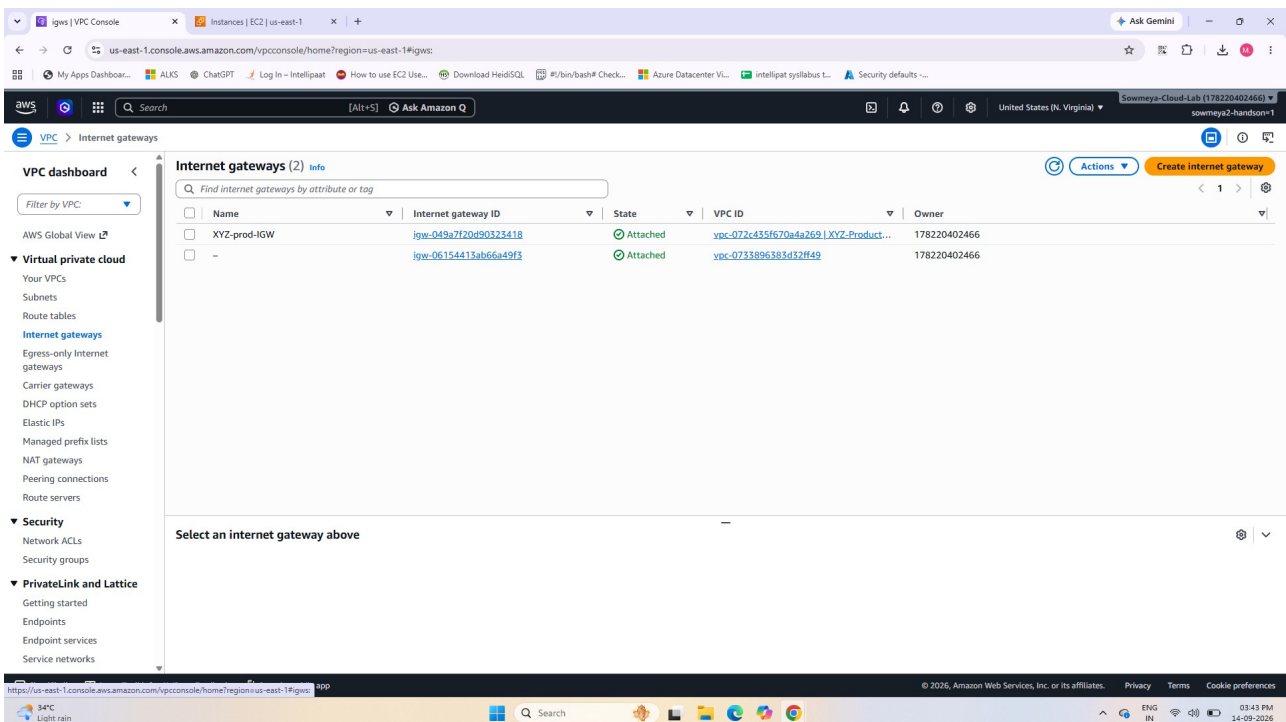


Figure 3: XYZ-prod-IGW attached to XYZ-Production-VPC

Step 4: Create the NAT Gateway

A NAT gateway lets private instances start outbound connections to the internet (for updates, APIs, etc.) while still blocking any inbound connection from the internet.

1. Go to **NAT gateways** → **Create NAT gateway**. Name: **XYZ-prod-NAT**.

- Subnet: **XYZ-prod-web-public** (the NAT must sit in a public subnet). Connectivity type: **Public**.
- Click **Allocate Elastic IP**, then **Create NAT gateway**. Wait until the state is **Available**.

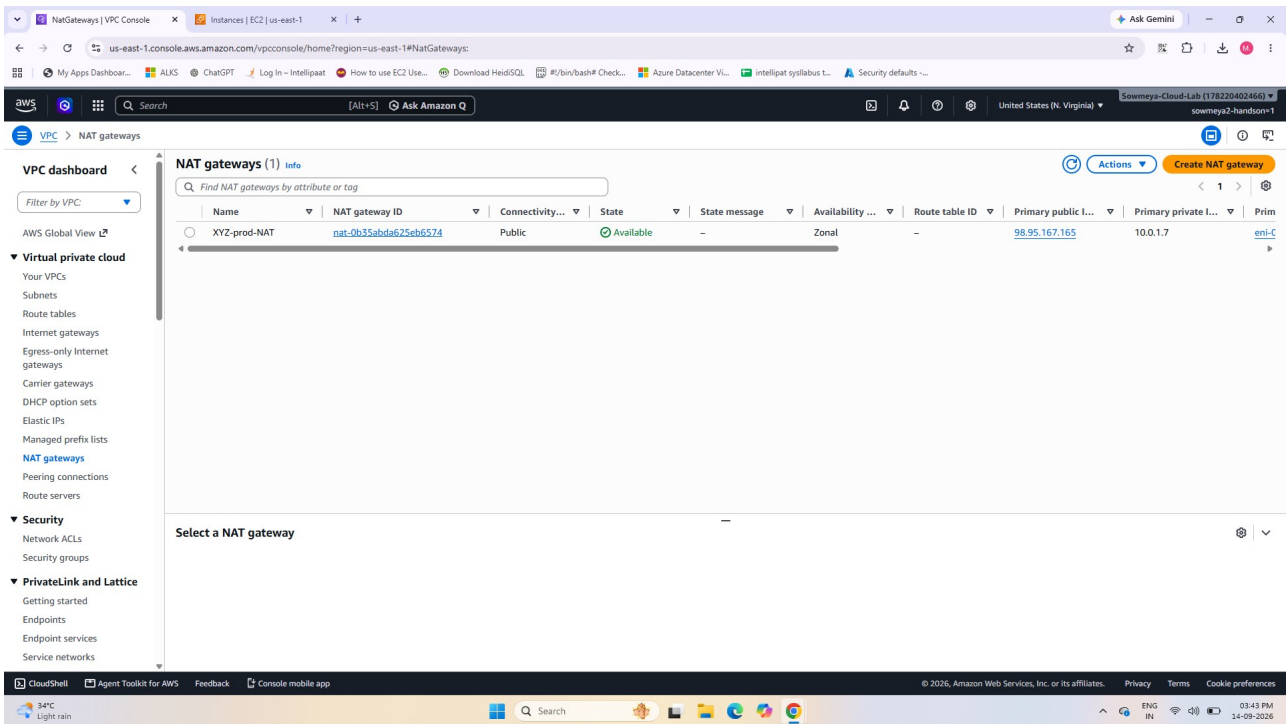


Figure 4: XYZ-prod-NAT Available – public IP 98.95.167.165, private IP 10.0.1.7 (in the web subnet)

Step 5: Create the route tables

Three custom route tables control which subnets can reach the internet:

Route table	Routes	Associated subnets
XYZ-prod-public-RT	10.0.0.0/16 → local, 0.0.0.0/0 → XYZ-prod-IGW	XYZ-prod-web-public
XYZ-prod-private-internet-RT	10.0.0.0/16 → local, 0.0.0.0/0 → XYZ-prod-NAT	XYZ-prod-app1-private, XYZ-prod-dbcache-private
XYZ-prod-private-no-internet-RT	10.0.0.0/16 → local (peering route added in Step 19)	XYZ-prod-app2-private, XYZ-prod-db-private

- Go to **Route tables** → **Create route table**, select **XYZ-Production-VPC** and create the three tables above.

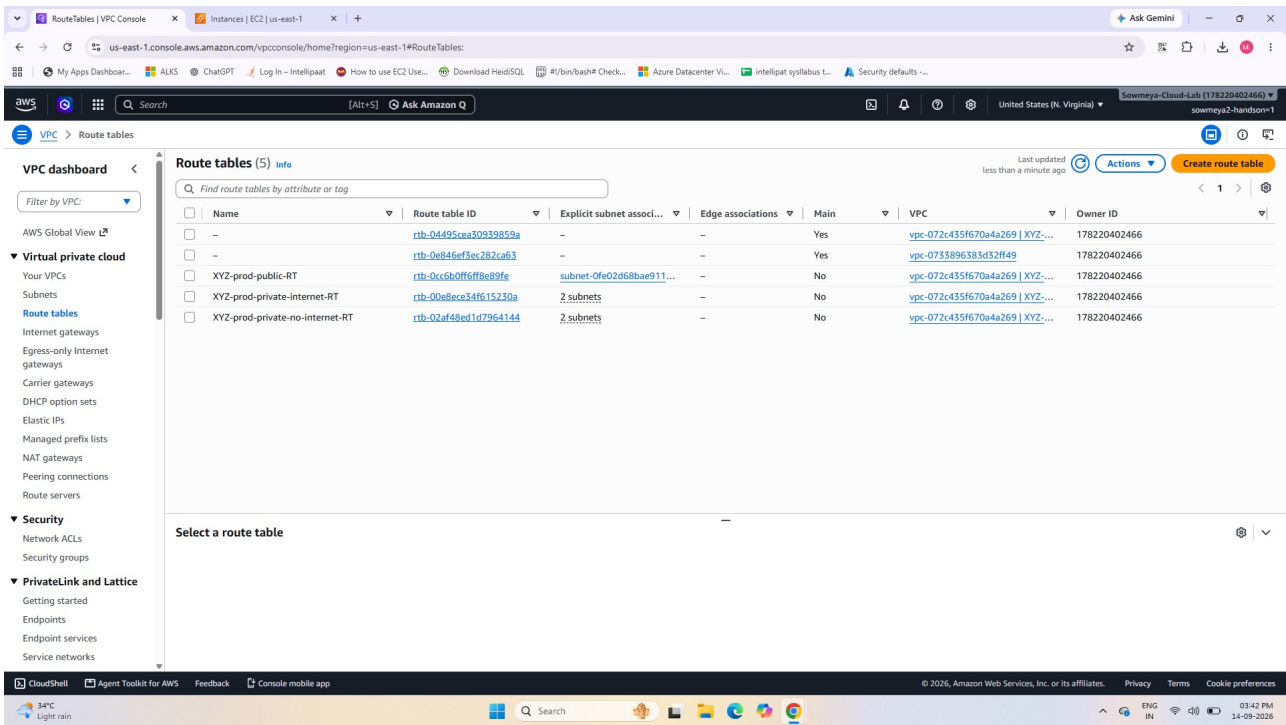


Figure 5: Route tables for XYZ-Production-VPC

5a. Public route table

1. Open XYZ-prod-public-RT → Routes → Edit routes → Add route: Destination 0.0.0.0/0, Target Internet Gateway → XYZ-prod-IGW. Save.
2. Subnet associations → Edit subnet associations → select XYZ-prod-web-public. Save.

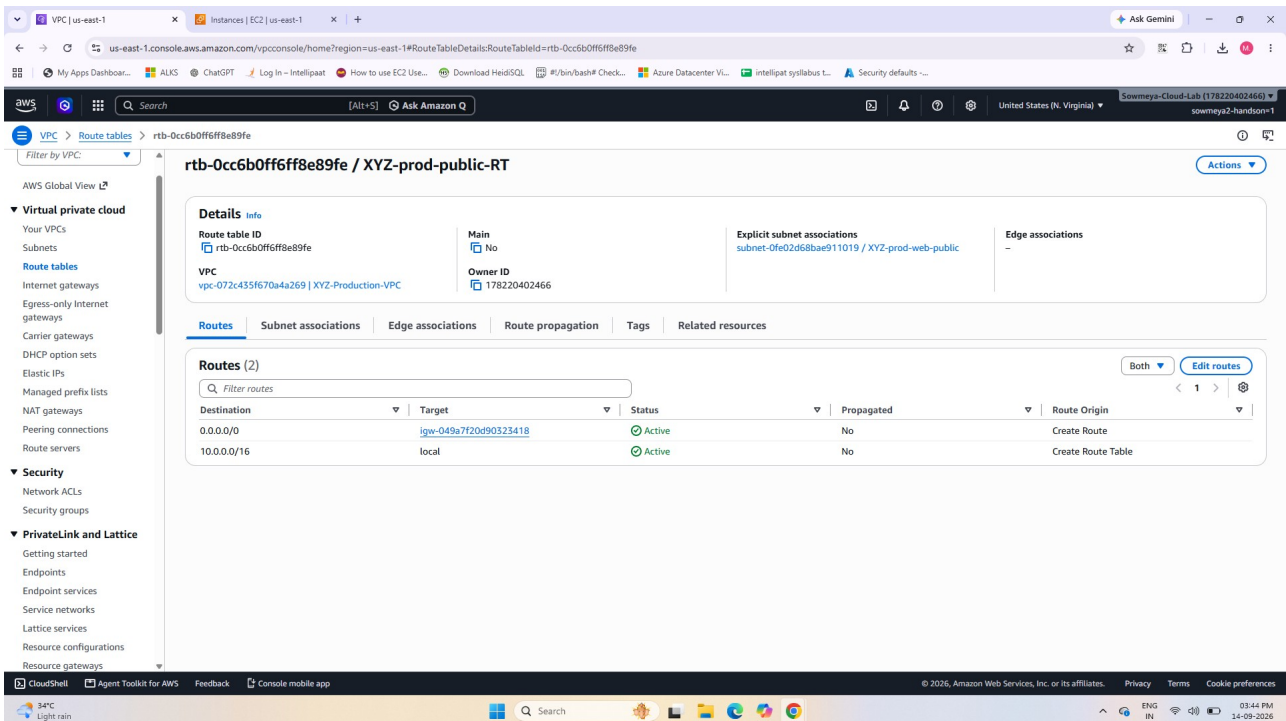


Figure 6: XYZ-prod-public-RT routes – 0.0.0.0/0 to the Internet Gateway

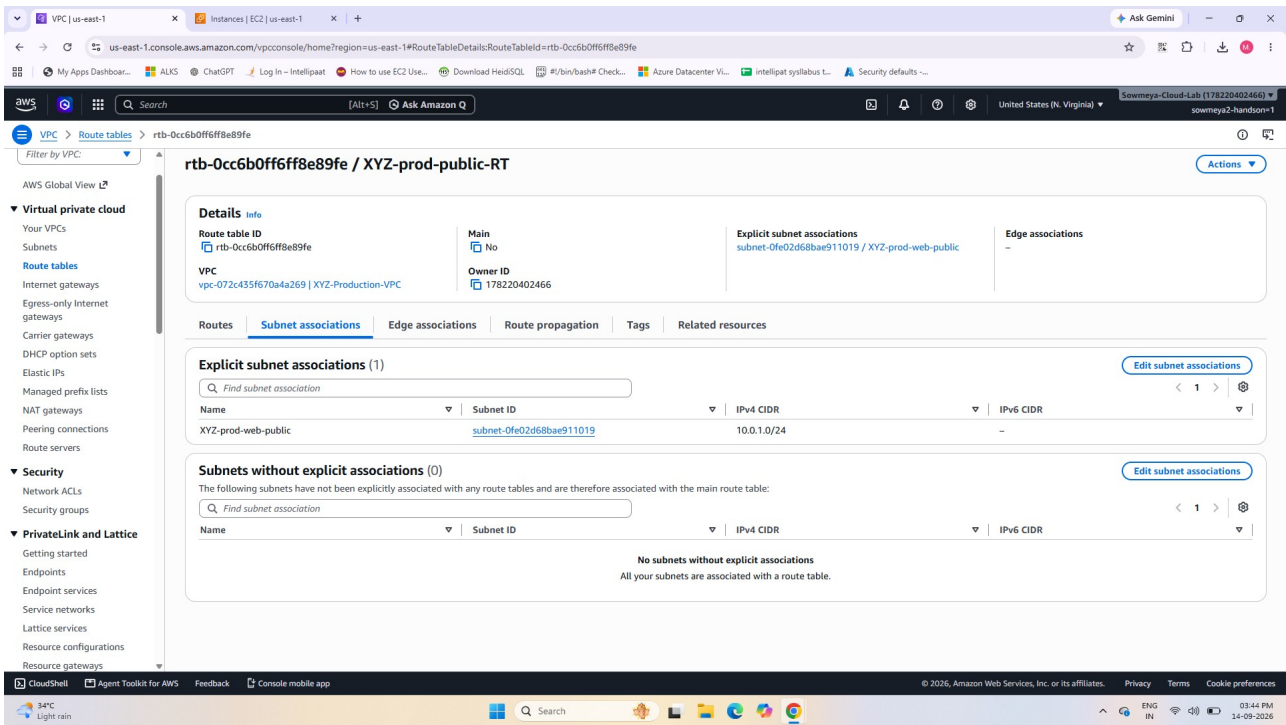


Figure 7: XYZ-prod-public-RT associated with XYZ-prod-web-public

5b. Private route table with internet (via NAT)

1. Open **XYZ-prod-private-internet-RT** → **Edit routes** → **Add route**: Destination **0.0.0.0/0**, Target **NAT Gateway** → **XYZ-prod-NAT**. Save.
2. **Edit subnet associations** → select **XYZ-prod-app1-private** and **XYZ-prod-dbcache-private**. Save.

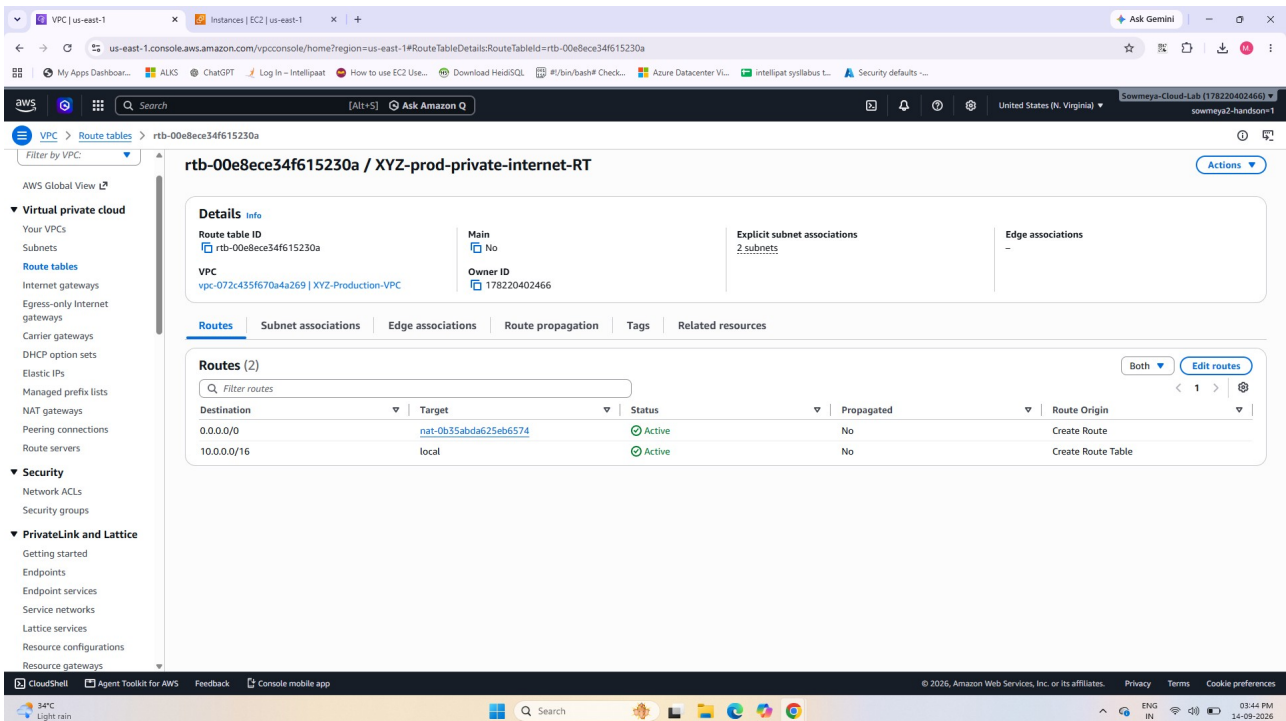


Figure 8: XYZ-prod-private-internet-RT routes – 0.0.0.0/0 to the NAT gateway

Task 4 met: app1 and dbcache are the only private subnets associated with this route table, so they are the only private subnets that can send internet requests.

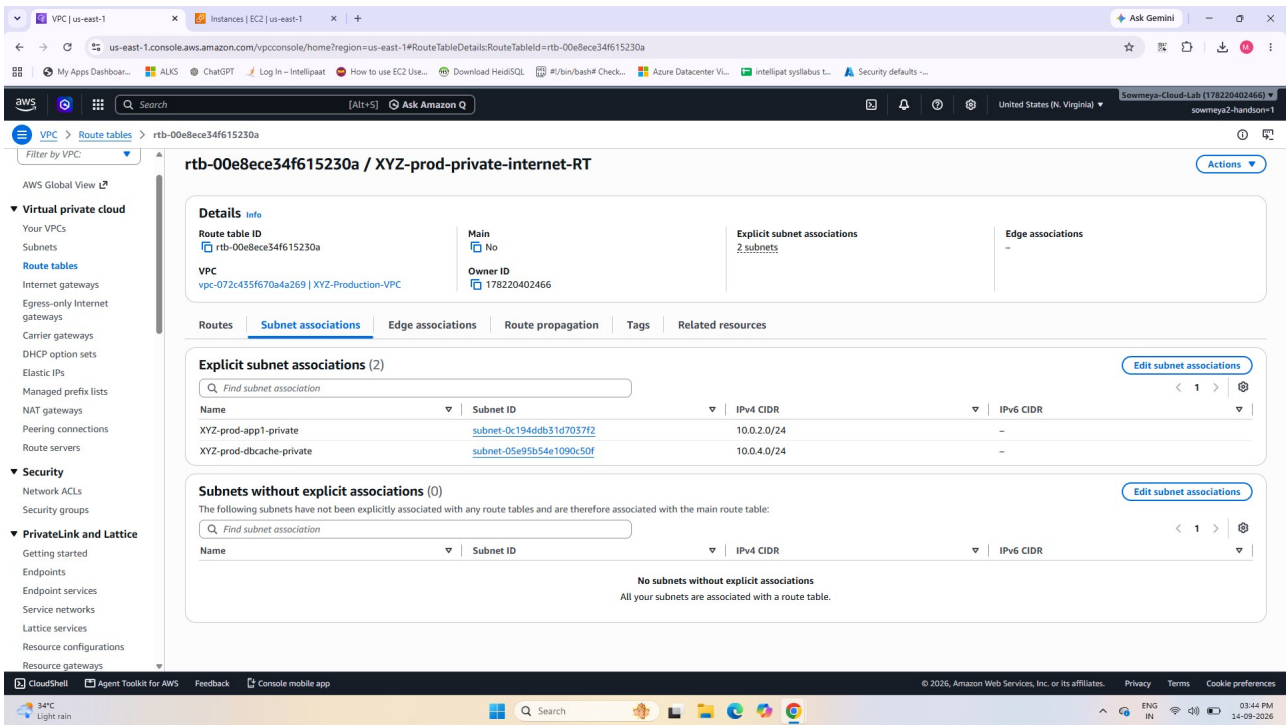


Figure 9: XYZ-prod-private-internet-RT associated with app1 and dbcache subnets

5c. Private route table without internet

1. Leave XYZ-prod-private-no-internet-RT with only the local route (no 0.0.0.0/0 route).
2. Edit subnet associations → select XYZ-prod-db-private and XYZ-prod-app2-private. Save.

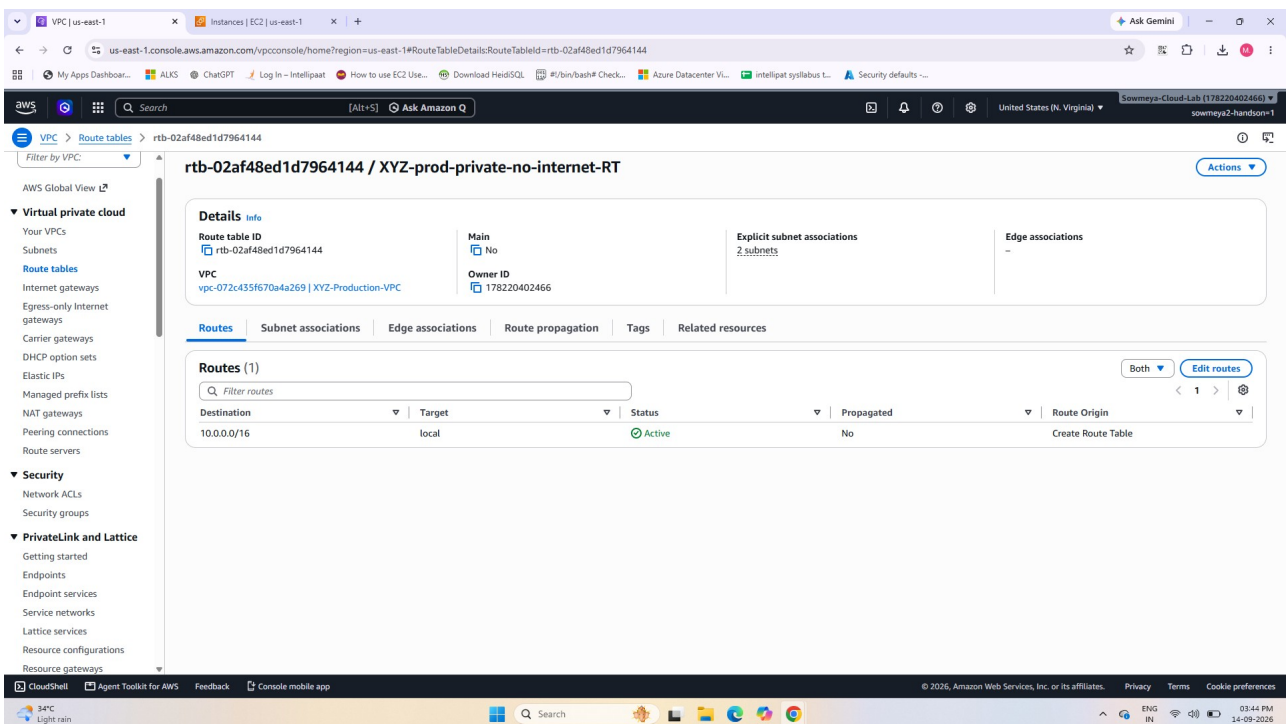


Figure 10: XYZ-prod-private-no-internet-RT – local route only

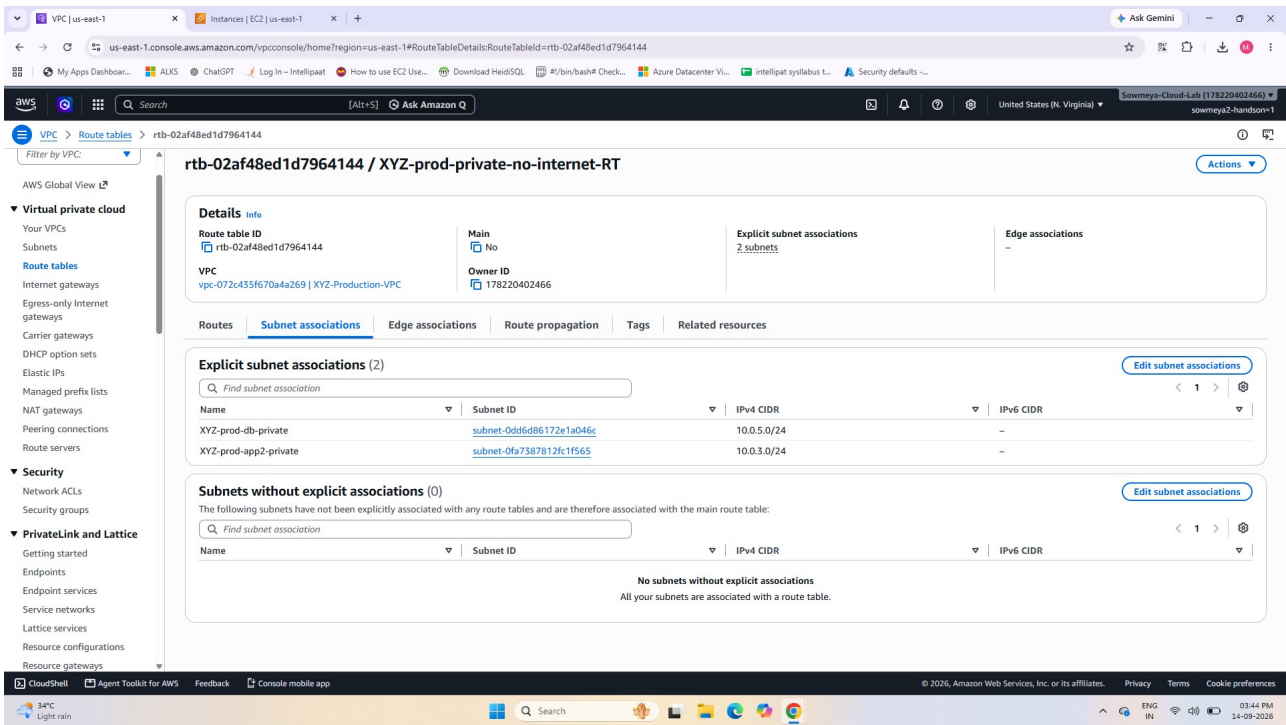


Figure 11: XYZ-prod-private-no-internet-RT associated with db and app2 subnets

Step 6: Create the security groups

Security groups are stateful firewalls attached to each instance. One group was created per tier:

Security group	Description	Used by
XYZ-prod-web-SG	Security group for production web server	XYZ-prod-web
XYZ-prod-app-SG	Security group for production application	XYZ-prod-app1, XYZ-prod-app2
XYZ-prod-dbcache-SG	Security group for production DB cache	XYZ-prod-dbcache
XYZ-prod-db-SG	Security group for production database	XYZ-prod-db

1. Go to **EC2 → Security Groups → Create security group** and select **XYZ-Production-VPC**.
2. Create the four groups above, adding inbound rules so that each tier only accepts traffic it needs (for example HTTP/HTTPS from the internet for web, and traffic only from the tier in front for the private tiers).

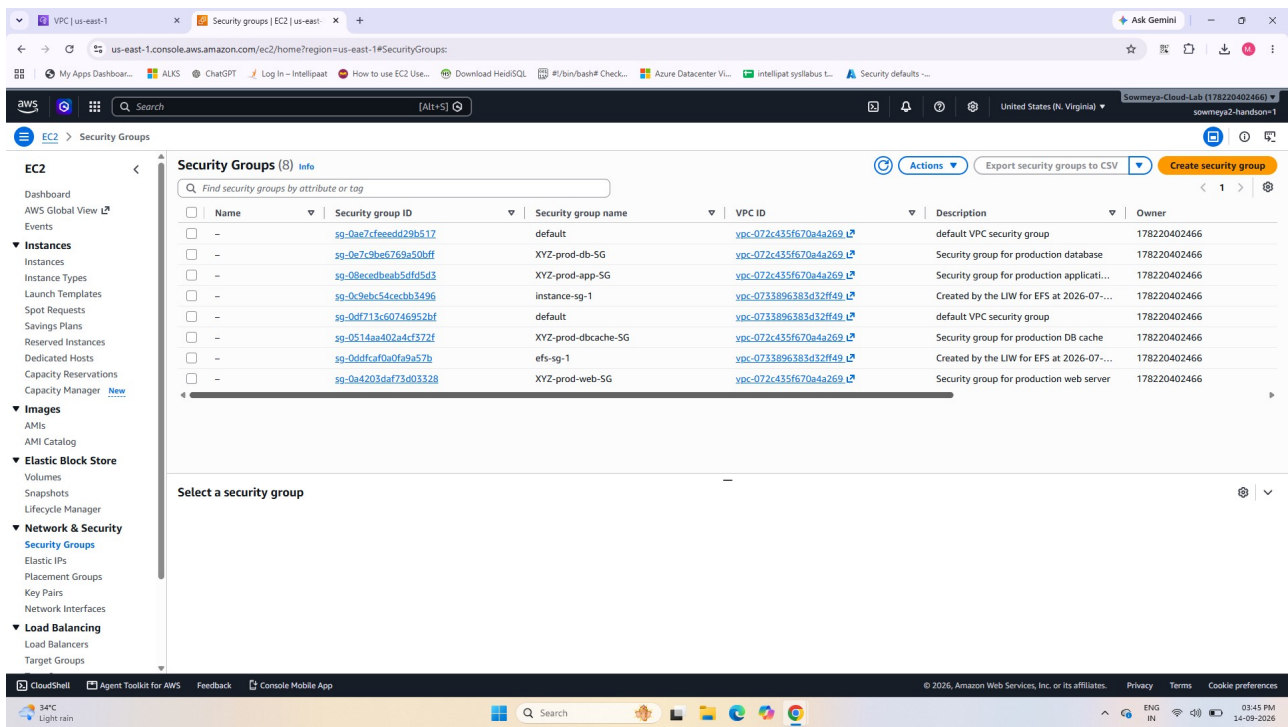


Figure 12: Production security groups – web, app, dbcache and db

Step 7: Create the network ACLs

Network ACLs are stateless firewalls applied at the subnet level. The production VPC uses one NACL per layer:

Network ACL	Associated with	Rules
XYZ-prod-web-NACL	XYZ-prod-web-public	2 inbound, 2 outbound
XYZ-prod-app-NACL	2 subnets (application tier)	2 inbound, 2 outbound
XYZ-prod-data-NACL	2 subnets (data tier)	2 inbound, 2 outbound

1. Go to **VPC → Network ACLs → Create network ACL**, select **XYZ-Production-VPC** and create the three NACLs.
2. For each NACL, edit **Inbound rules** and **Outbound rules** (rule 100 plus the default * deny rule).
3. Open **Subnet associations → Edit subnet associations** and attach the matching subnets.

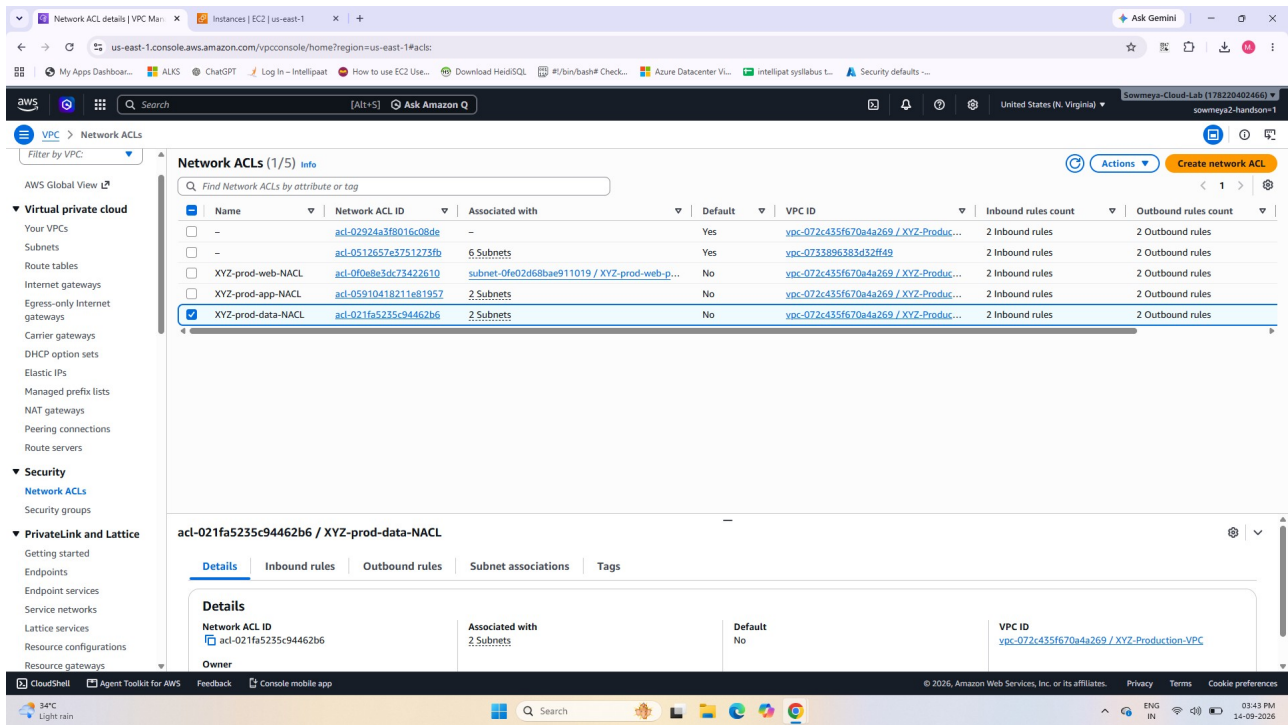


Figure 13: Production NACLs – XYZ-prod-web-NACL, XYZ-prod-app-NACL and XYZ-prod-data-NACL

Step 8: Launch an instance in every subnet

1. Go to **EC2 → Launch instances**. Instance type **t3.micro**, Network **XYZ-Production-VPC**.
2. Launch one instance per subnet, name it after the subnet, and attach the matching security group:

Instance name	Subnet	Security group	Public IP
XYZ-prod-web	XYZ-prod-web-public	XYZ-prod-web-SG	54.160.35.246
XYZ-prod-app1	XYZ-prod-app1-private	XYZ-prod-app-SG	–
XYZ-prod-app2	XYZ-prod-app2-private	XYZ-prod-app-SG	–
XYZ-prod-dbcache	XYZ-prod-dbcache-private	XYZ-prod-dbcache-SG	–
XYZ-prod-db	XYZ-prod-db-private	XYZ-prod-db-SG	–

The screenshot displays the AWS Management Console for the us-east-1 region, specifically the EC2 Instances page. Five production instances are listed, all in a 'Running' state. The instance 'XYZ-prod-web' is the only one with a public IPv4 address assigned. The console also shows health checks, availability zones, and various management links for each instance.

Name	Instance ID	Instance state	Instance type	Status check	Availability Zone	Public IPv4 DNS	Public IPv4 ...	Elastic IP	IPv6 IPs
XYZ-prod-abc	i-02ca02166c6de0899	Running	t3.micro	3/3 checks passed	us-east-1a	-	-	-	-
XYZ-prod-app2	i-0351d67fb4eeff09c	Running	t3.micro	3/3 checks passed	us-east-1a	-	-	-	-
XYZ-prod-web	i-092446600dc41294d	Running	t3.micro	3/3 checks passed	us-east-1a	-	54.160.35.246	-	-
XYZ-prod-db	i-04718e488c6944c9	Running	t3.micro	3/3 checks passed	us-east-1a	-	-	-	-
XYZ-prod-app1	i-035ab6c7955b3b638	Running	t3.micro	3/3 checks passed	us-east-1a	-	-	-	-

All instances

- Instance alarm states:** In alarm: None. [View in CloudWatch](#)
- Service health:** EC2 service status: Operating normally. [View Service Health Dashboard](#)
- Health events affecting EC2 instances:** Open events: None. Upcoming events: None. [View AWS Health Dashboard](#)

Figure 14: Five production instances running – only XYZ-prod-web has a public IPv4 address

3. Development Network (2-Tier Architecture)

The development VPC is a simpler 2-tier design: a public web subnet and a private db subnet. Only the web subnet is allowed to reach the internet.

Step 9: Create the Development VPC

1. In the VPC console click **Create VPC → VPC only**. Name tag: **XYZ-Development-VPC**.
2. IPv4 CIDR block: **20.0.0.0/16** (must not overlap the production 10.0.0.0/16, otherwise peering is not possible). Click **Create VPC**.

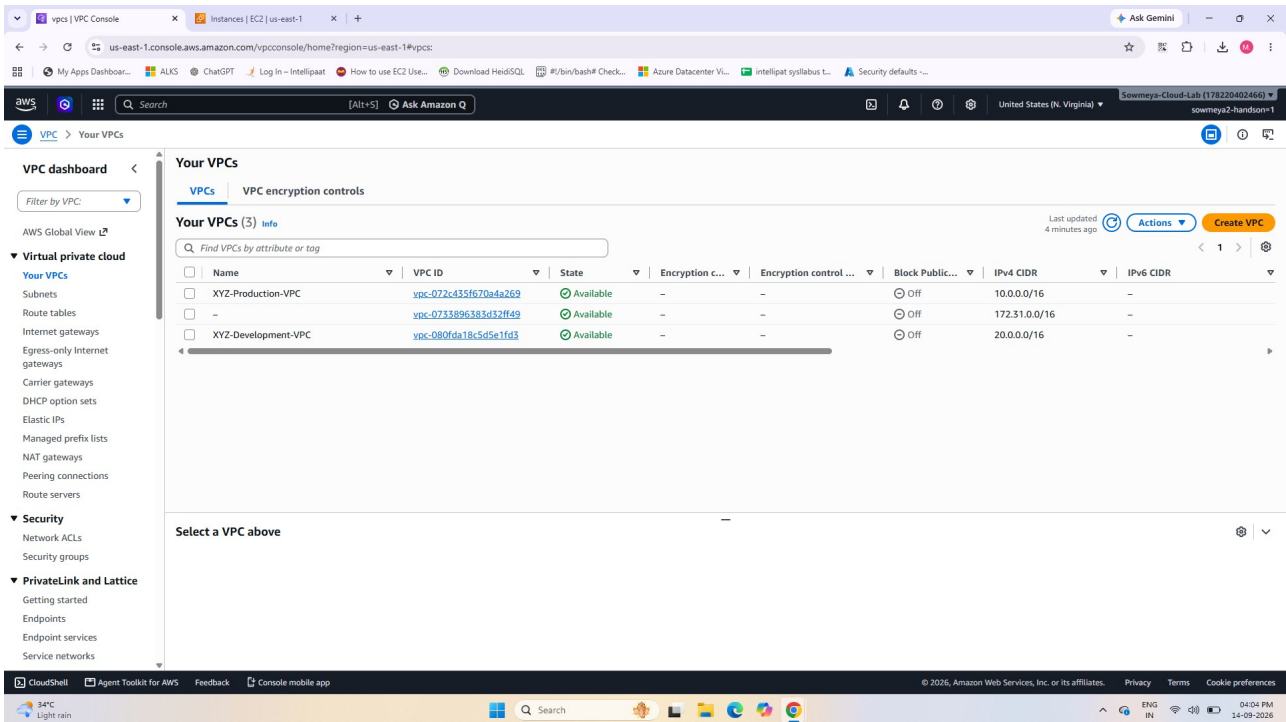


Figure 15: XYZ-Development-VPC created with CIDR 20.0.0.0/16

Step 10: Create the web and db subnets

1. **Subnets → Create subnet**, VPC **XYZ-Development-VPC**, Availability Zone **us-east-1a**.
2. Subnet 1: **XYZ-dev-web** – **20.0.1.0/24**.
3. Subnet 2: **XYZ-dev-db** – **20.0.2.0/24**.

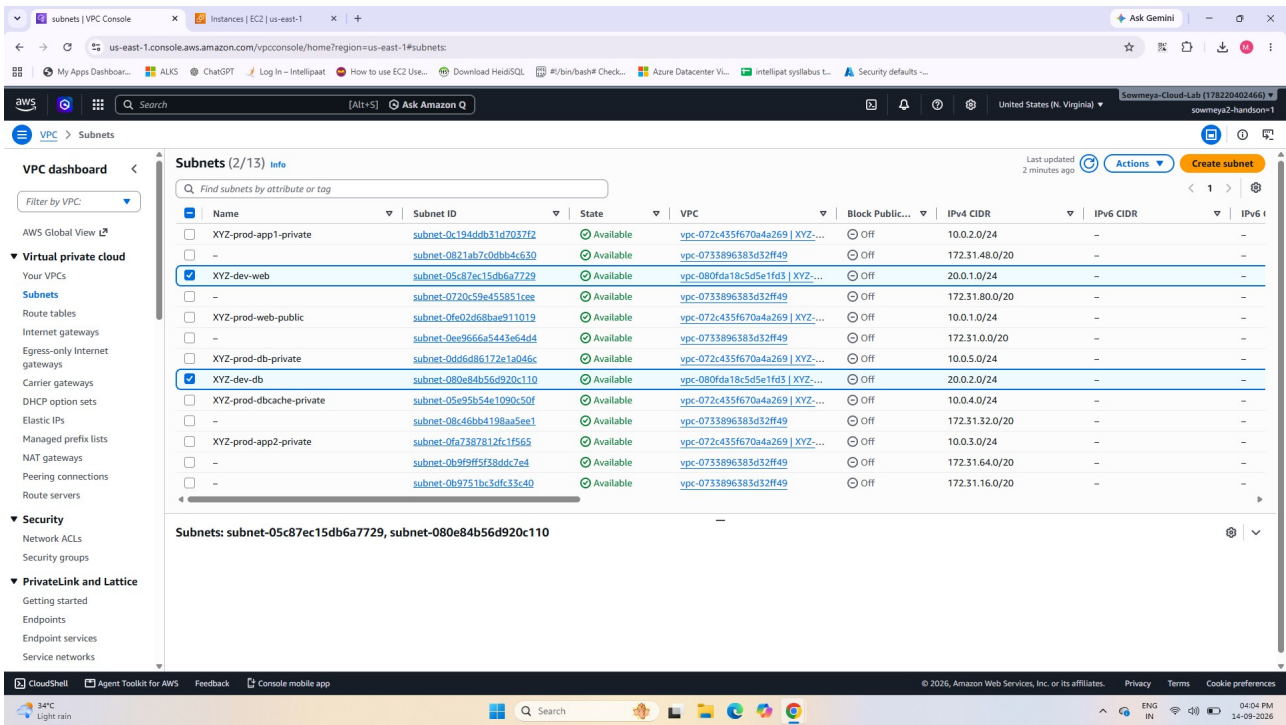


Figure 16: Development subnets XYZ-dev-web (20.0.1.0/24) and XYZ-dev-db (20.0.2.0/24)

Step 11: Enable auto-assign public IP on the web subnet

1. Select **XYZ-dev-web** → **Actions** → **Edit subnet settings**.
2. Tick **Enable auto-assign public IPv4 address** and click **Save**.

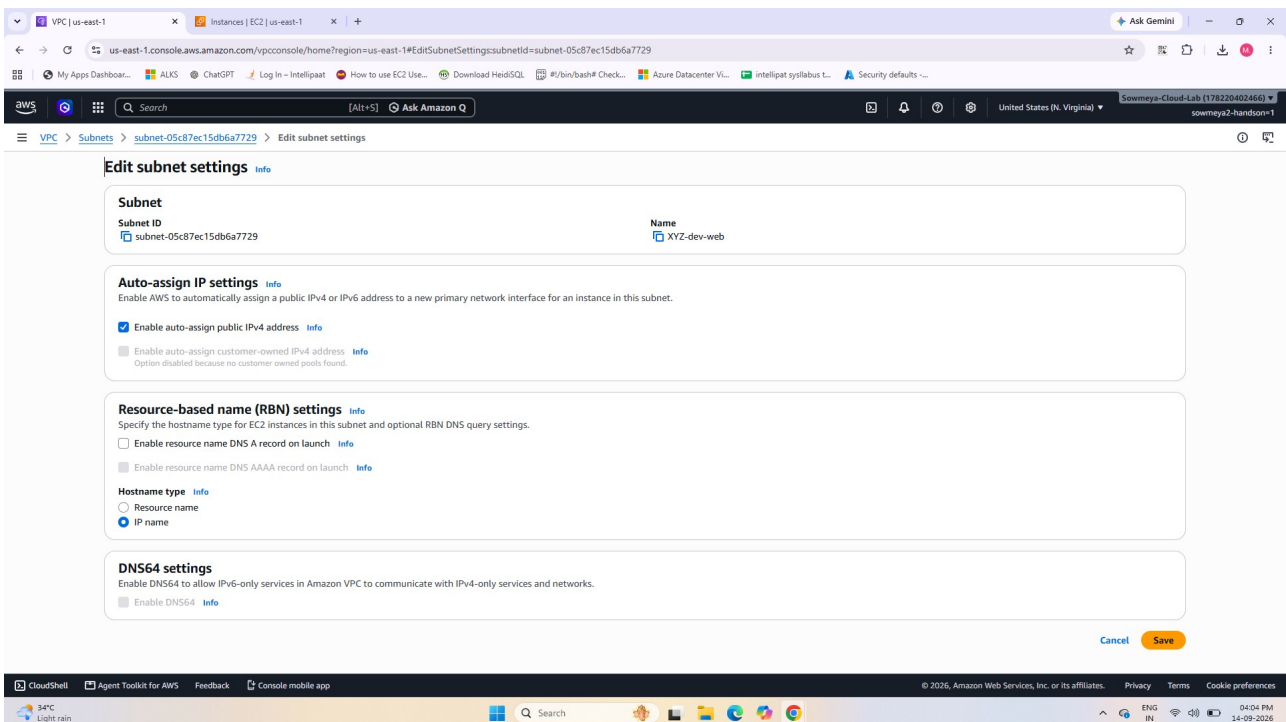


Figure 17: Auto-assign public IPv4 enabled on XYZ-dev-web

Step 12: Create and attach the Internet Gateway

1. **Internet gateways** → **Create internet gateway**. Name: **XYZ-dev-IGW**.
2. **Actions** → **Attach to VPC** → **XYZ-Development-VPC**.

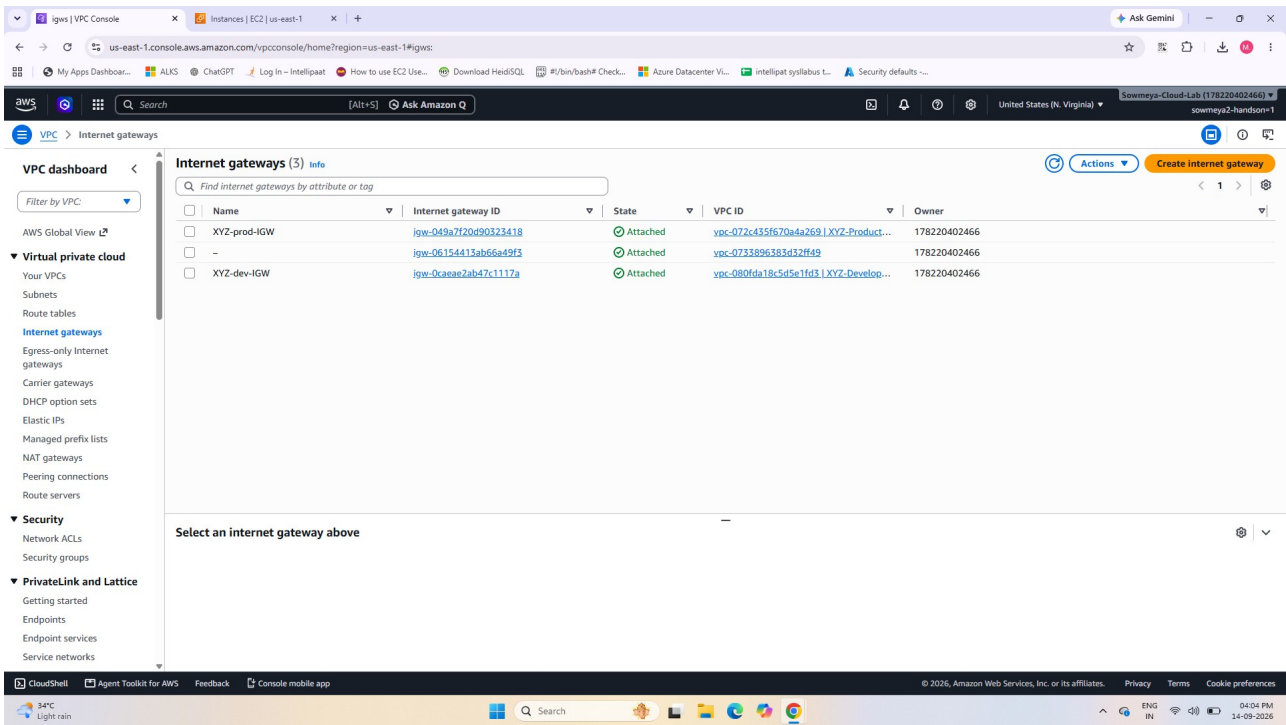


Figure 18: XYZ-dev-IGW attached to XYZ-Development-VPC

Step 13: Create the route tables

1. Create XYZ-dev-public-RT and XYZ-dev-private-RT in XYZ-Development-VPC.

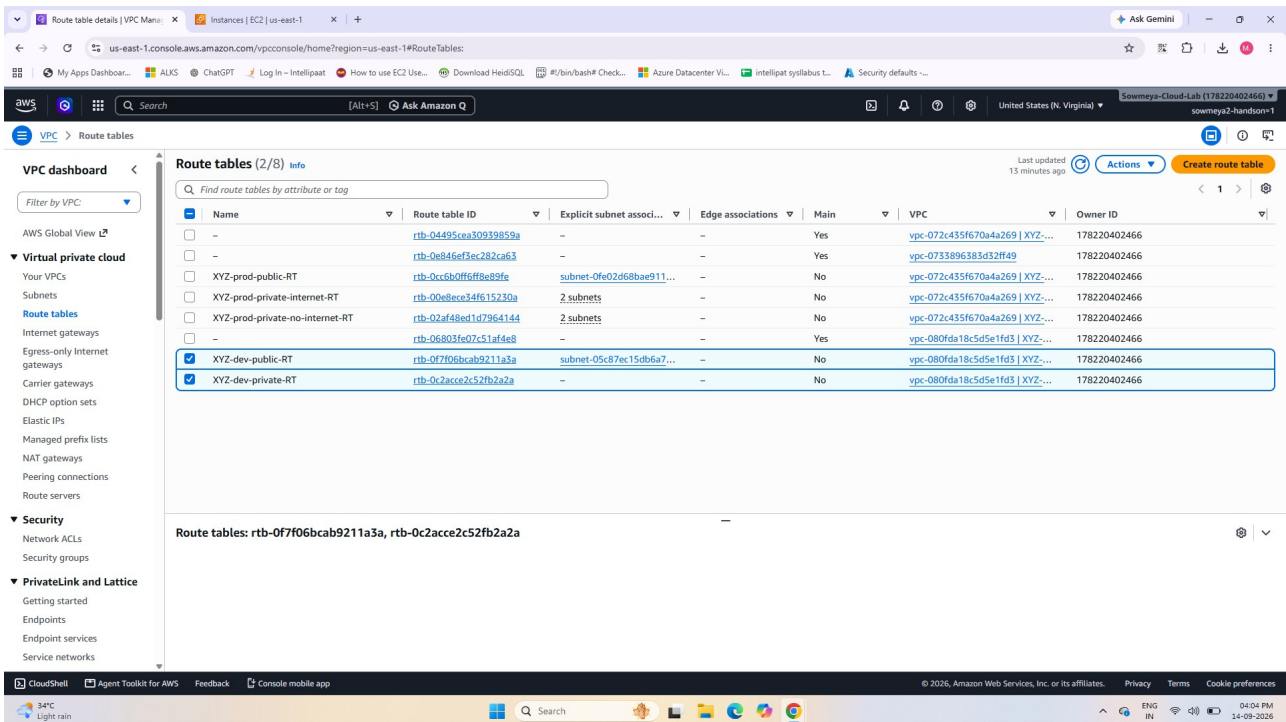


Figure 19: Development route tables XYZ-dev-public-RT and XYZ-dev-private-RT

13a. Public route table

1. XYZ-dev-public-RT → Edit routes → Add route: 0.0.0.0/0 → XYZ-dev-IGW. Save.
2. Edit subnet associations → select XYZ-dev-web. Save.

The screenshot shows the AWS Management Console for the VPC `rtb-0f7f06bcab9211a3a` (XYZ-dev-public-RT). The **Routes** tab is selected, displaying a table of routes:

Destination	Target	Status	Propagated	Route Origin
0.0.0.0/0	igw-0caae2ab47c1117a	Active	No	Create Route
20.0.0.0/16	local	Active	No	Create Route Table

The details section shows the route table ID `rtb-0f7f06bcab9211a3a`, VPC `vpc-080fda18c5d5e1fd3` (XYZ-Development-VPC), and owner ID `178220402466`. It also lists explicit subnet associations: `subnet-05c87ec15db6a7729` / XYZ-dev-web.

Figure 20: XYZ-dev-public-RT routes – 0.0.0.0/0 to XYZ-dev-IGW

The screenshot shows the AWS Management Console for the VPC `rtb-0f7f06bcab9211a3a` (XYZ-dev-public-RT). The **Subnet associations** tab is selected, displaying a table of explicit subnet associations:

Name	Subnet ID	IPv4 CIDR	IPv6 CIDR
XYZ-dev-web	subnet-05c87ec15db6a7729	20.0.1.0/24	-

Below this, the **Subnets without explicit associations** section shows:

Name	Subnet ID	IPv4 CIDR	IPv6 CIDR
XYZ-dev-db	subnet-080e84b56d920c110	20.0.2.0/24	-

The details section shows the route table ID `rtb-0f7f06bcab9211a3a`, VPC `vpc-080fda18c5d5e1fd3` (XYZ-Development-VPC), and owner ID `178220402466`. It also lists explicit subnet associations: `subnet-05c87ec15db6a7729` / XYZ-dev-web.

Figure 21: XYZ-dev-public-RT associated with XYZ-dev-web

13b. Private route table

1. Keep XYZ-dev-private-RT with only the local route (no internet route).
2. Edit subnet associations → select XYZ-dev-db. Save.

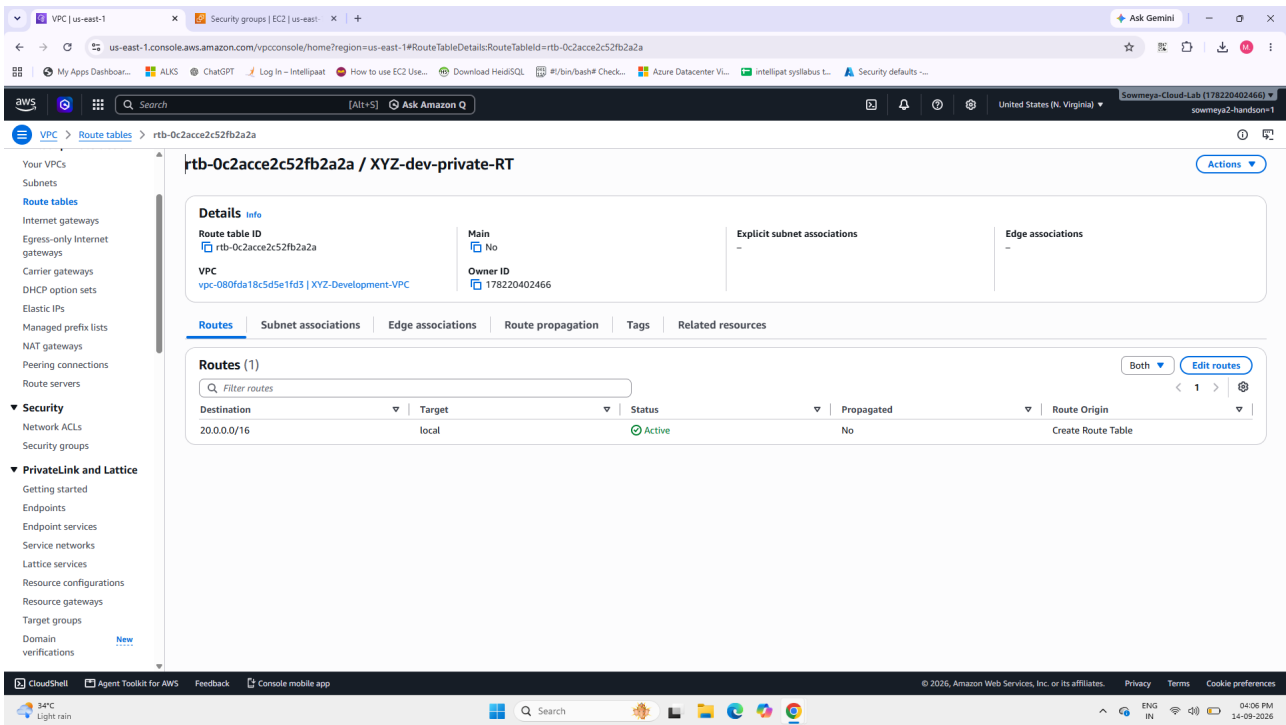


Figure 22: XYZ-dev-private-RT – local route 20.0.0.0/16 only

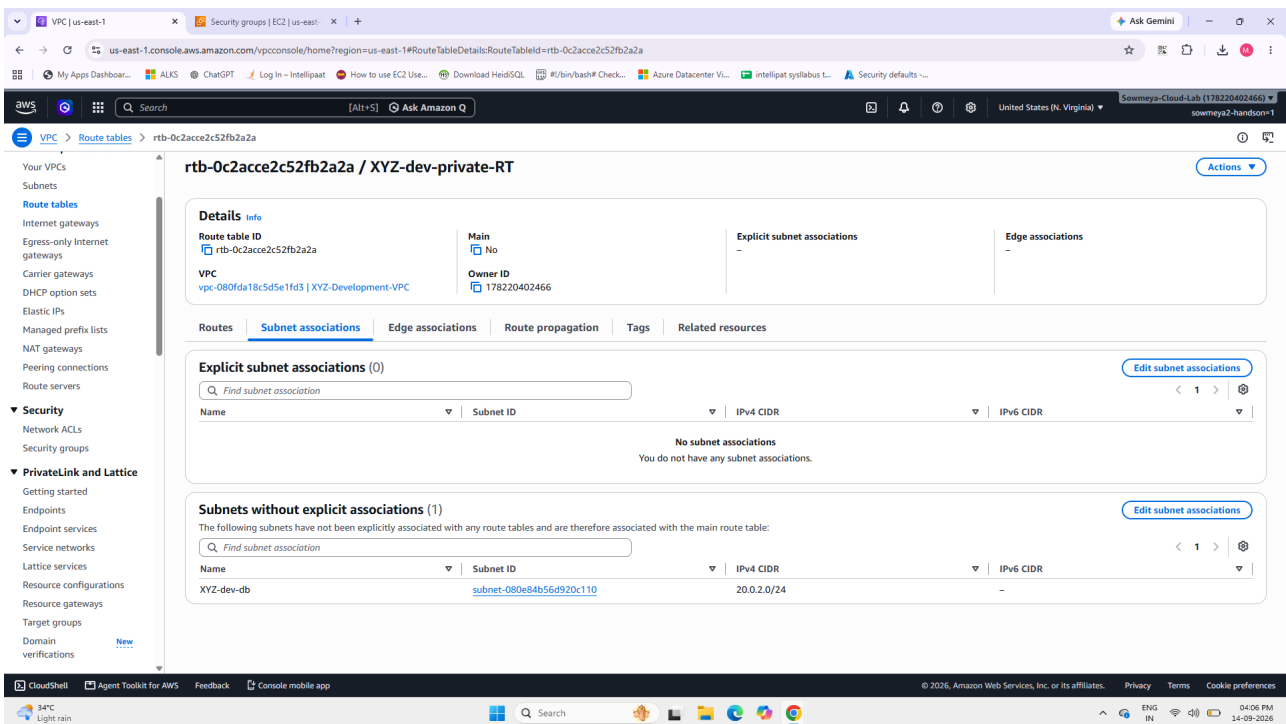


Figure 23: XYZ-dev-private-RT subnet associations

Note: In Figure 23, XYZ-dev-db is still listed under **Subnets without explicit associations**, so it is using the VPC main route table. The main table also has no internet route, so dev-db is still private. Associate XYZ-dev-db with XYZ-dev-private-RT so the peering route in Step 19 applies to it.

Task 2 met: Only XYZ-dev-web is on a route table with a 0.0.0.0/0 route to the Internet Gateway, so only the web subnet can send internet requests.

Step 14: Create the network ACLs

1. **VPC → Network ACLs → Create network ACL: XYZ-dev-web-NACL and XYZ-dev-db-NACL in XYZ-Development-VPC.**
2. Inbound and outbound rule **100**: All traffic, 0.0.0.0/0, **Allow** (the default * rule denies everything else).
3. **Subnet associations → Edit subnet associations: XYZ-dev-web-NACL → XYZ-dev-web, XYZ-dev-db-NACL → XYZ-dev-db.**

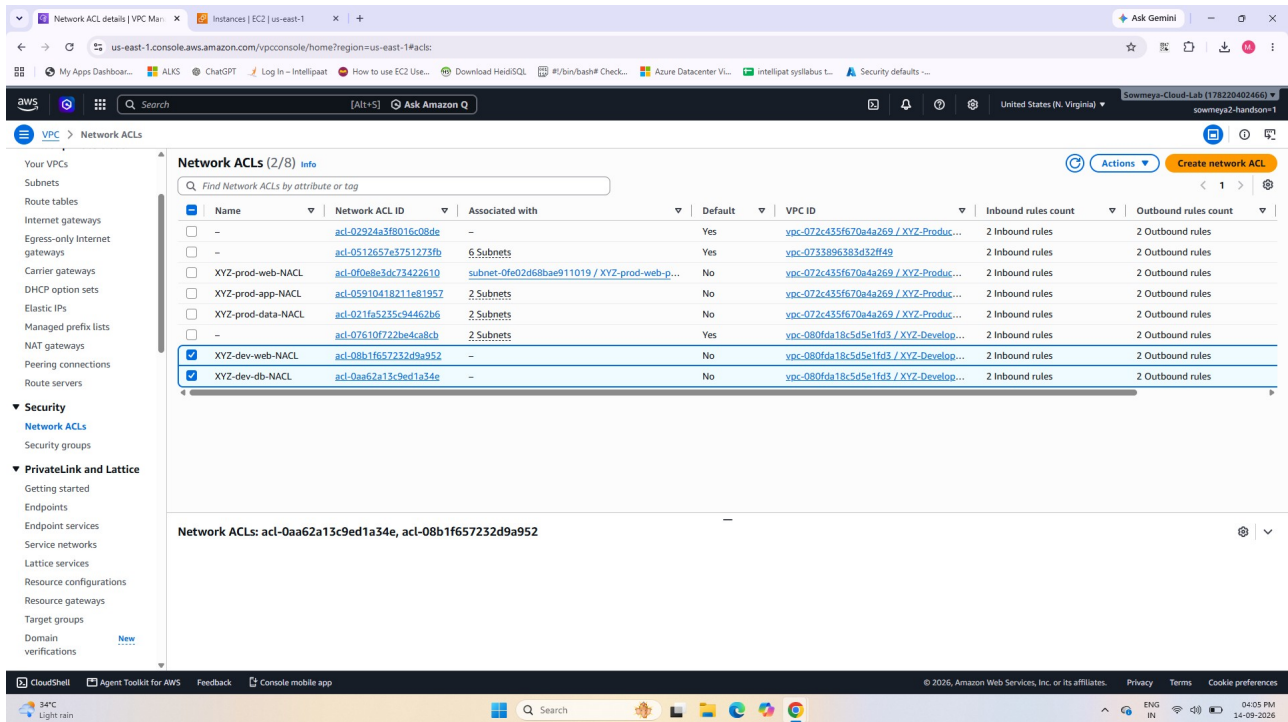


Figure 24: Development NACLs XYZ-dev-web-NACL and XYZ-dev-db-NACL

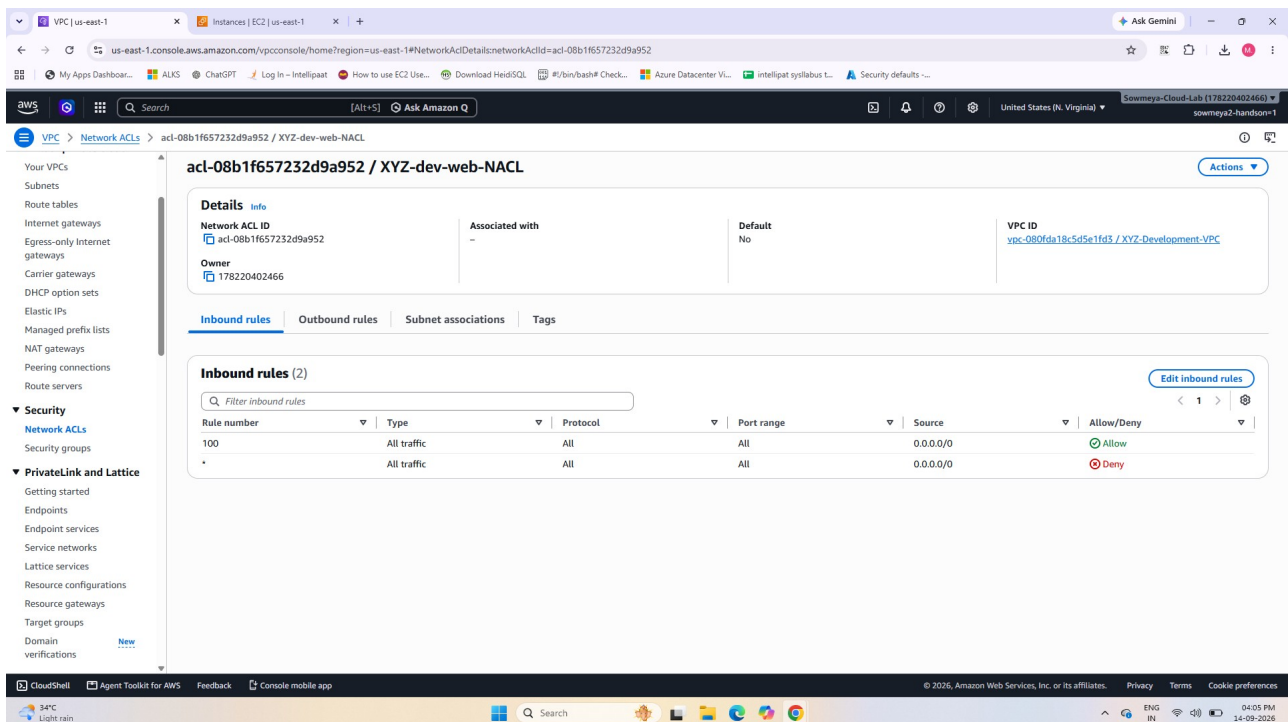


Figure 25: XYZ-dev-web-NACL inbound rules – rule 100 allow all, * deny

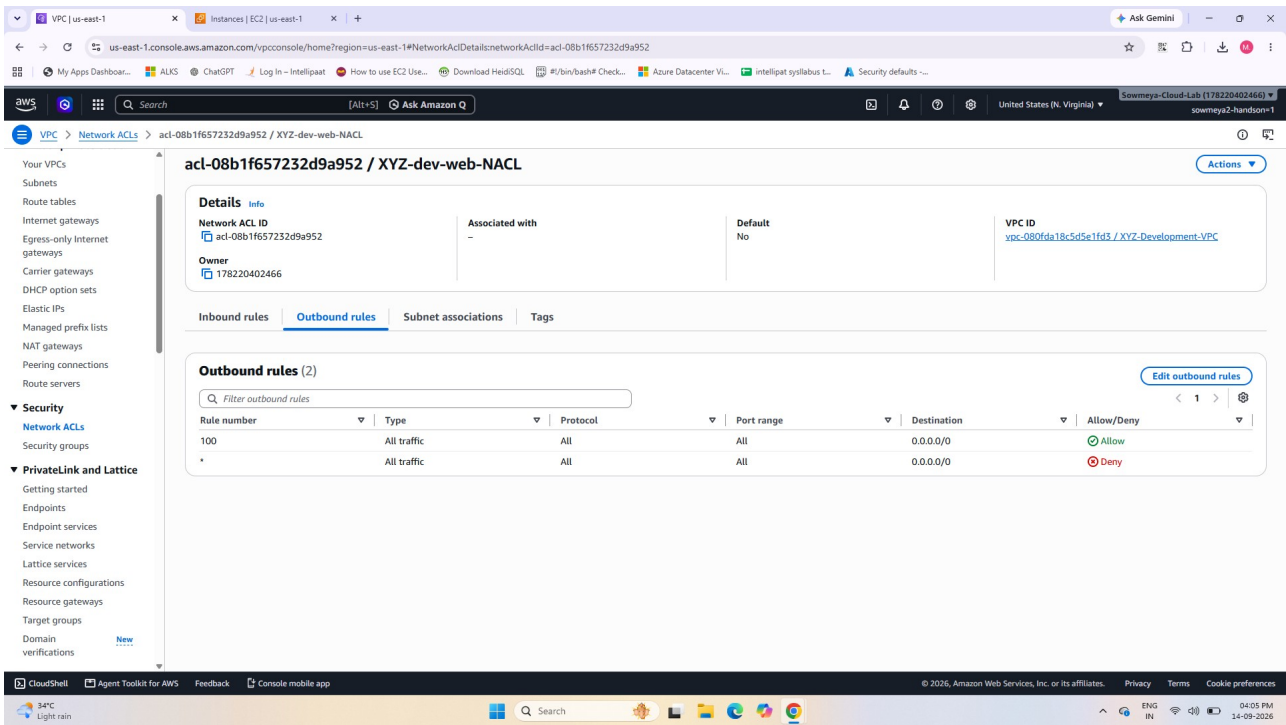


Figure 26: XYZ-dev-web-NACL outbound rules – rule 100 allow all, * deny

Step 15: Create the security groups

1. EC2 → Security Groups → Create security group in XYZ-Development-VPC.
2. XYZ-dev-web-SG – Security group for development web server.
3. XYZ-dev-db-SG – Security group for development database (inbound rule added in Step 20).

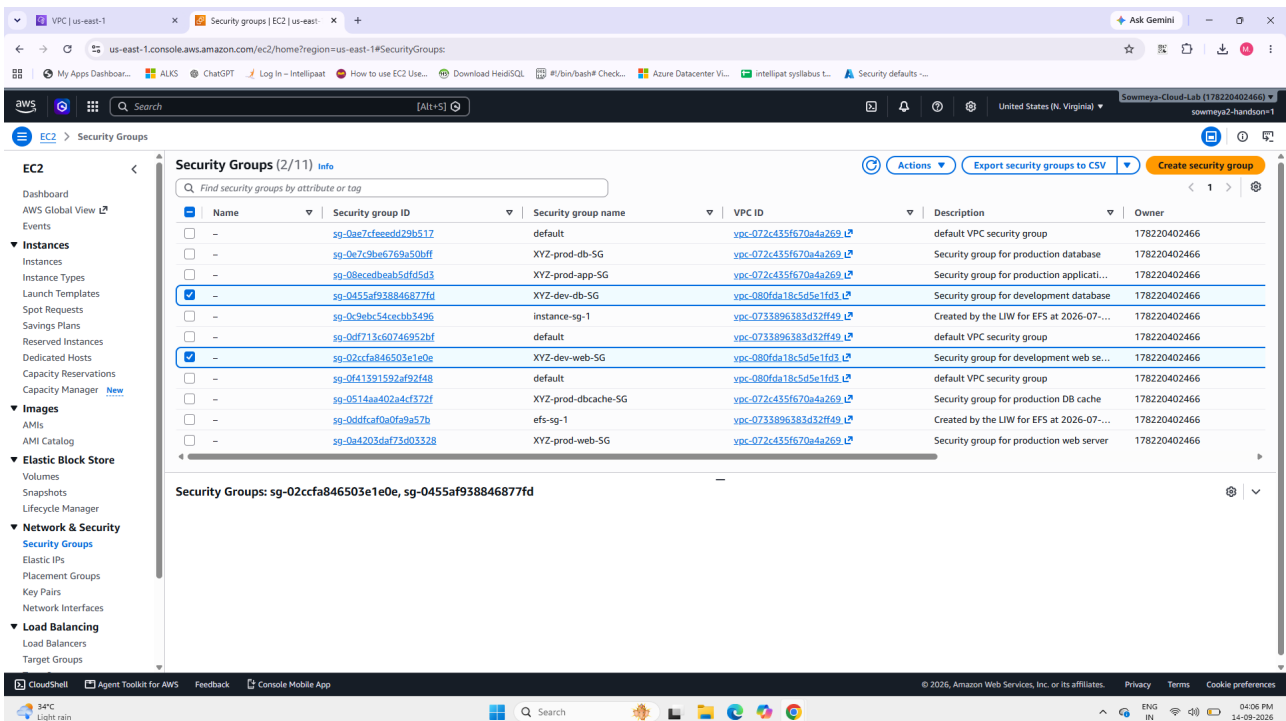
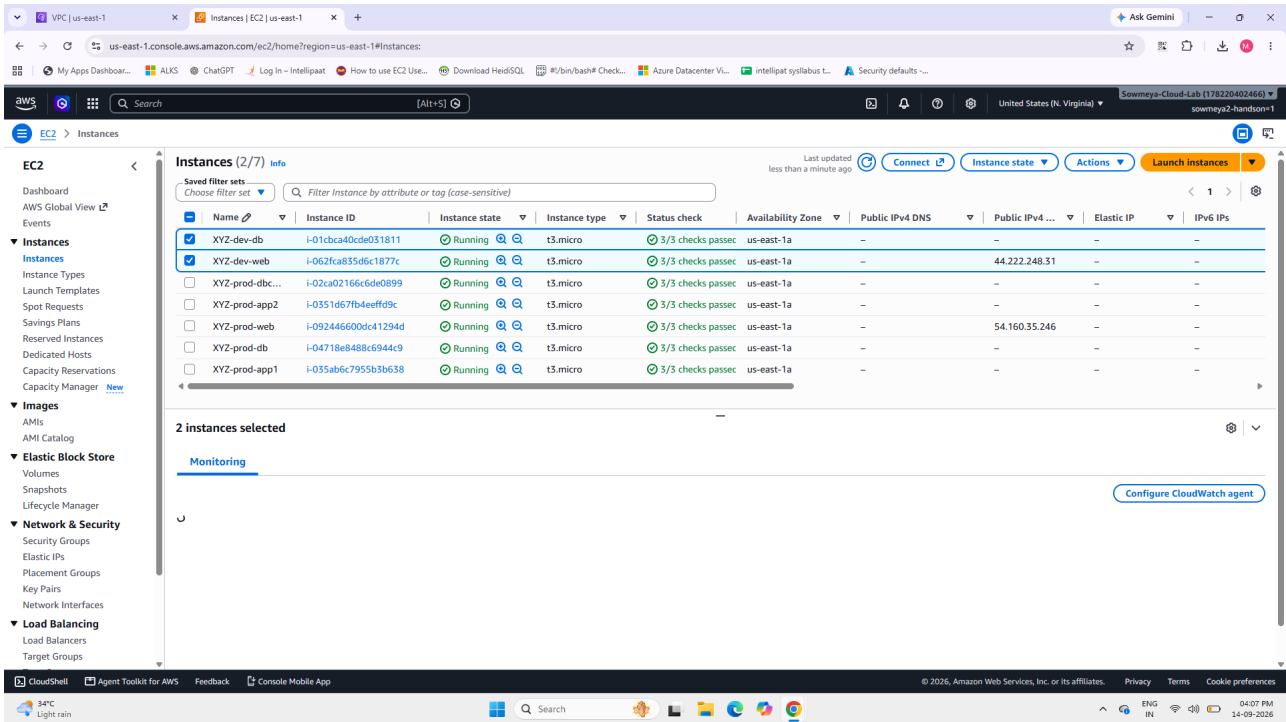


Figure 27: Development security groups XYZ-dev-web-SG and XYZ-dev-db-SG

Step 16: Launch the development instances

1. Launch XYZ-dev-web (t3.micro) in subnet XYZ-dev-web with XYZ-dev-web-SG – it receives public IP 44.222.248.31.

2. Launch XYZ-dev-db (t3.micro) in subnet XYZ-dev-db with XYZ-dev-db-SG – no public IP.



The screenshot shows the AWS Management Console for the EC2 Instances page. The table lists 7 instances, all in the 'Running' state. The XYZ-dev-db instance has a public IPv4 address of 44.222.248.31. The XYZ-dev-web instance has a public IPv4 address of 54.160.35.246. The production instances have public IPv4 addresses of 54.160.35.246, 54.160.35.246, 54.160.35.246, 54.160.35.246, and 54.160.35.246 respectively.

Name	Instance ID	Instance state	Instance type	Status check	Availability Zone	Public IPv4 DNS	Public IPv4 ...	Elastic IP	IPv6 IPs
XYZ-dev-db	i-01cbca40cde031811	Running	t3.micro	3/3 checks passed	us-east-1a	-	-	-	-
XYZ-dev-web	i-062fca835d5c1877c	Running	t3.micro	3/3 checks passed	us-east-1a	-	44.222.248.31	-	-
XYZ-prod-abc	i-02ca02166c6de0899	Running	t3.micro	3/3 checks passed	us-east-1a	-	-	-	-
XYZ-prod-app2	i-0351d67fb4eeff09c	Running	t3.micro	3/3 checks passed	us-east-1a	-	-	-	-
XYZ-prod-web	i-092446600dc41294d	Running	t3.micro	3/3 checks passed	us-east-1a	-	54.160.35.246	-	-
XYZ-prod-db	i-04718e8488c6944c9	Running	t3.micro	3/3 checks passed	us-east-1a	-	-	-	-
XYZ-prod-app1	i-035ab6c7955b3b638	Running	t3.micro	3/3 checks passed	us-east-1a	-	-	-	-

Figure 28: XYZ-dev-web and XYZ-dev-db running alongside the five production instances

4. VPC Peering and DB-to-DB Connection

VPC peering joins the two VPCs so that instances can talk to each other using private IP addresses, as if they were on the same network. Traffic flows only when both route tables point to the peering connection and the security groups allow it.

Step 17: Create the peering connection

1. Go to **VPC → Peering connections → Create peering connection**. Name: **XYZ-Prod-Dev-Peering**.
2. VPC ID (Requester): **XYZ-Production-VPC (10.0.0.0/16)**.
3. Account: **My account**. Region: **This Region (us-east-1)**. VPC ID (Accepter): **XYZ-Development-VPC (20.0.0.0/16)**.
4. Click **Create peering connection**, then select it and choose **Actions → Accept request**.

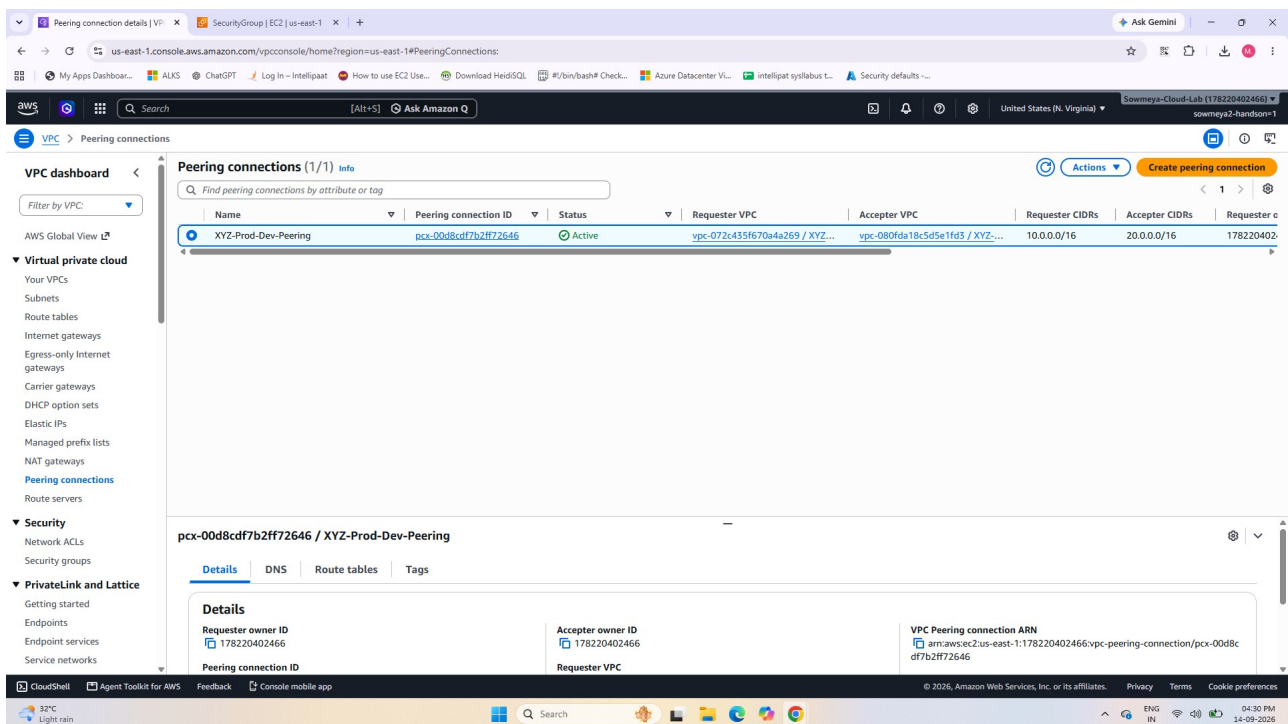


Figure 29: XYZ-Prod-Dev-Peering is Active – requester 10.0.0.0/16, acceptor 20.0.0.0/16

Step 18: Review the peering details

The details page confirms both VPCs, their CIDRs and the region. The Route tables tab lists every route table that sends traffic through this peering connection.

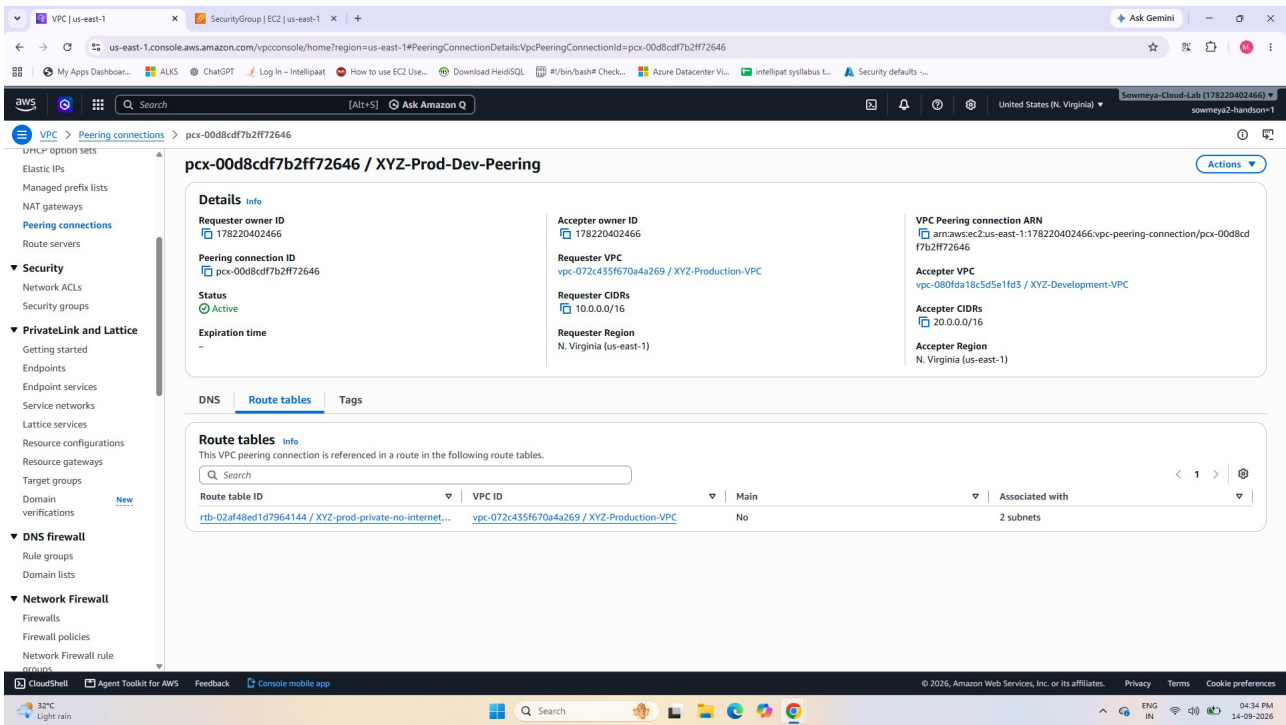


Figure 30: Peering connection details and the route tables that reference it

Step 19: Add routes for the peering connection

1. Production side: open **XYZ-prod-private-no-internet-RT** → **Edit routes** → **Add route**: Destination **20.0.0.0/16**, Target **Peering Connection** → **pcx-00d8cdf7b2ff72646**. Save.
2. Development side: open **XYZ-dev-private-RT** → **Edit routes** → **Add route**: Destination **10.0.0.0/16** (or 10.0.5.0/24 for the db subnet only), Target **pcx-00d8cdf7b2ff72646**. Save.

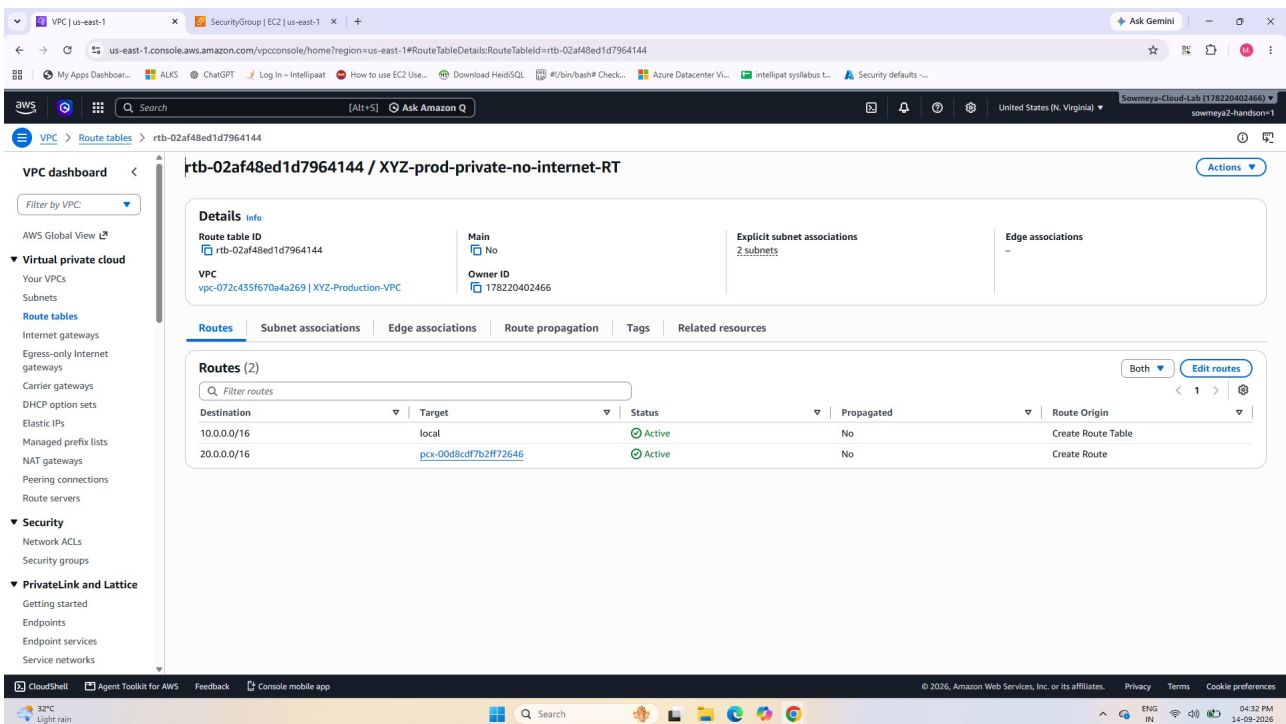


Figure 31: XYZ-prod-private-no-internet-RT – route 20.0.0.0/16 through the peering connection

Important: Routes are needed on both sides. Without the development route, replies from XYZ-dev-db cannot get back to the production VPC.

Step 20: Allow database traffic in the security groups

1. Open **XYZ-dev-db-SG** → **Edit inbound rules** → **Add rule**: Type **MYSQL/Aurora** (TCP 3306), Source **10.0.5.0/24** (production db subnet). Save.
2. Open **XYZ-prod-db-SG** → **Edit inbound rules** → **Add rule**: Type **MYSQL/Aurora** (TCP 3306), Source **20.0.2.0/24** (development db subnet). Save.

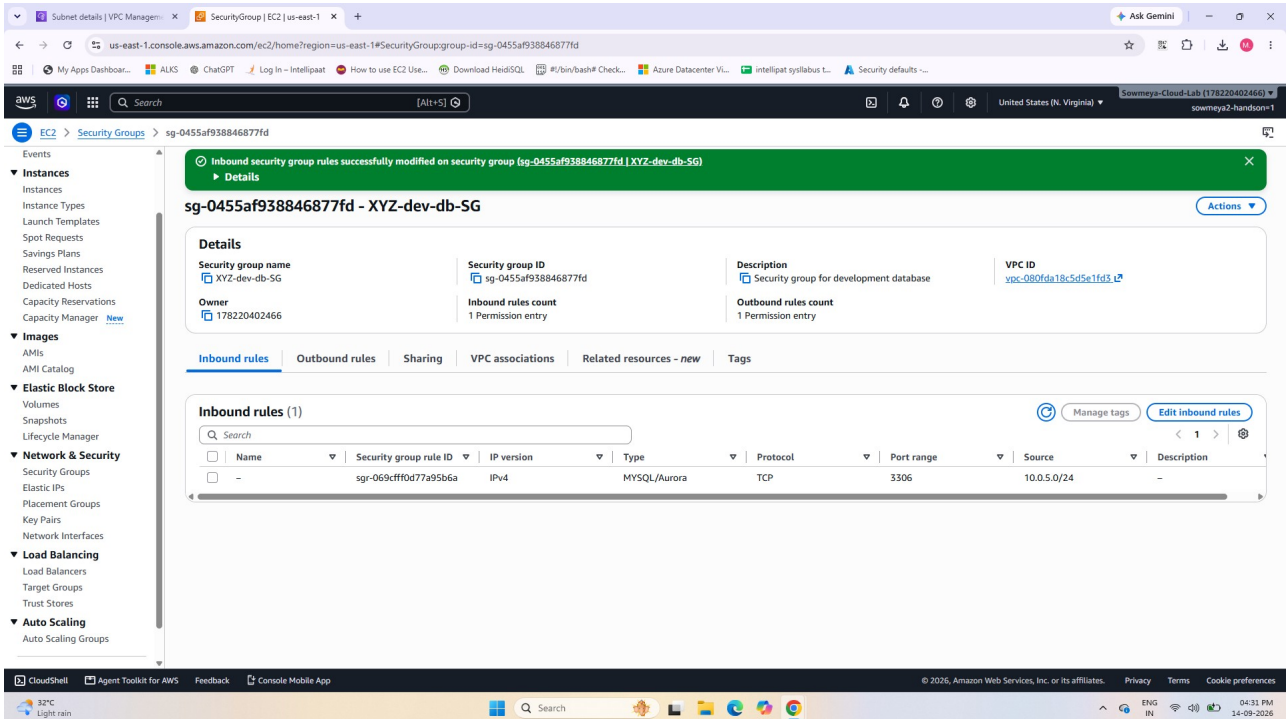


Figure 32: XYZ-dev-db-SG – MySQL 3306 allowed from the production db subnet 10.0.5.0/24

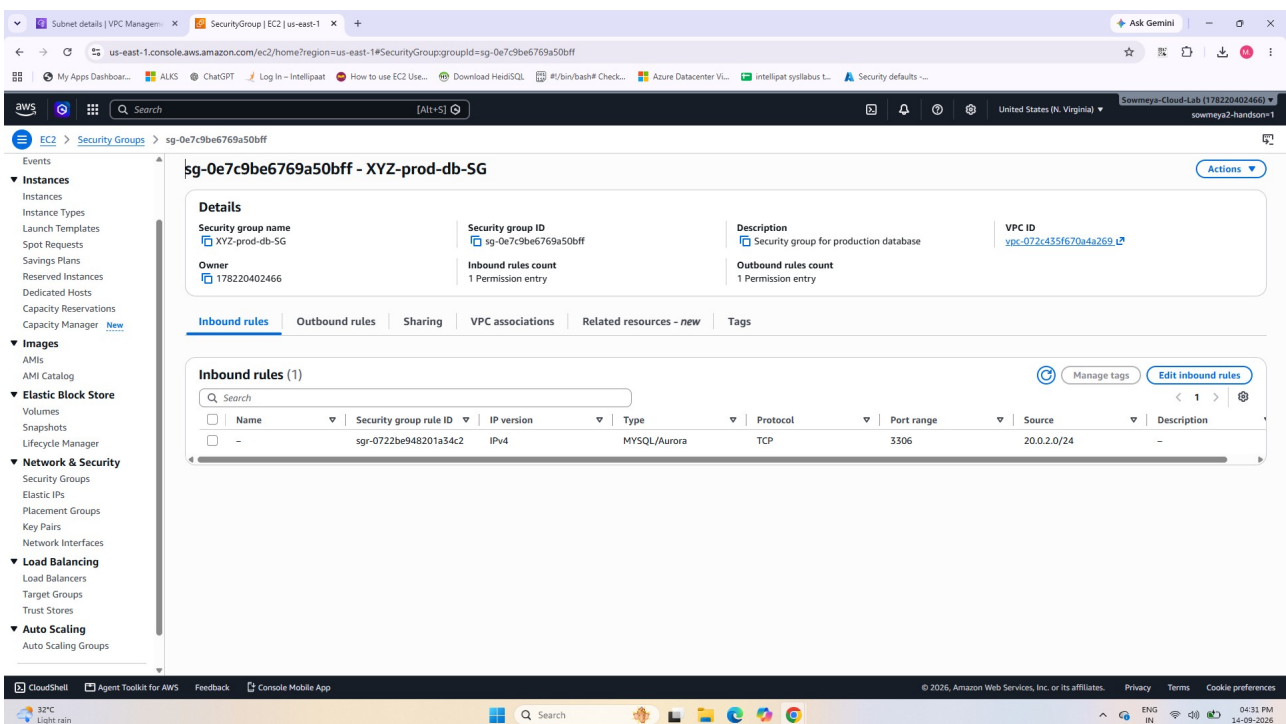


Figure 33: XYZ-prod-db-SG – MySQL 3306 allowed from the development db subnet 20.0.2.0/24

Result: Only the two db subnets can reach each other's MySQL port, so the link is limited to the database layer.

5. Final Verification – Resource Maps

The VPC Resource map shows how subnets, route tables and gateways are connected.

Production VPC resource map

All five subnets are in us-east-1a. The web subnet uses the public route table (Internet Gateway), app1 and dbcache use the NAT route table, and app2 and db use the no-internet route table.

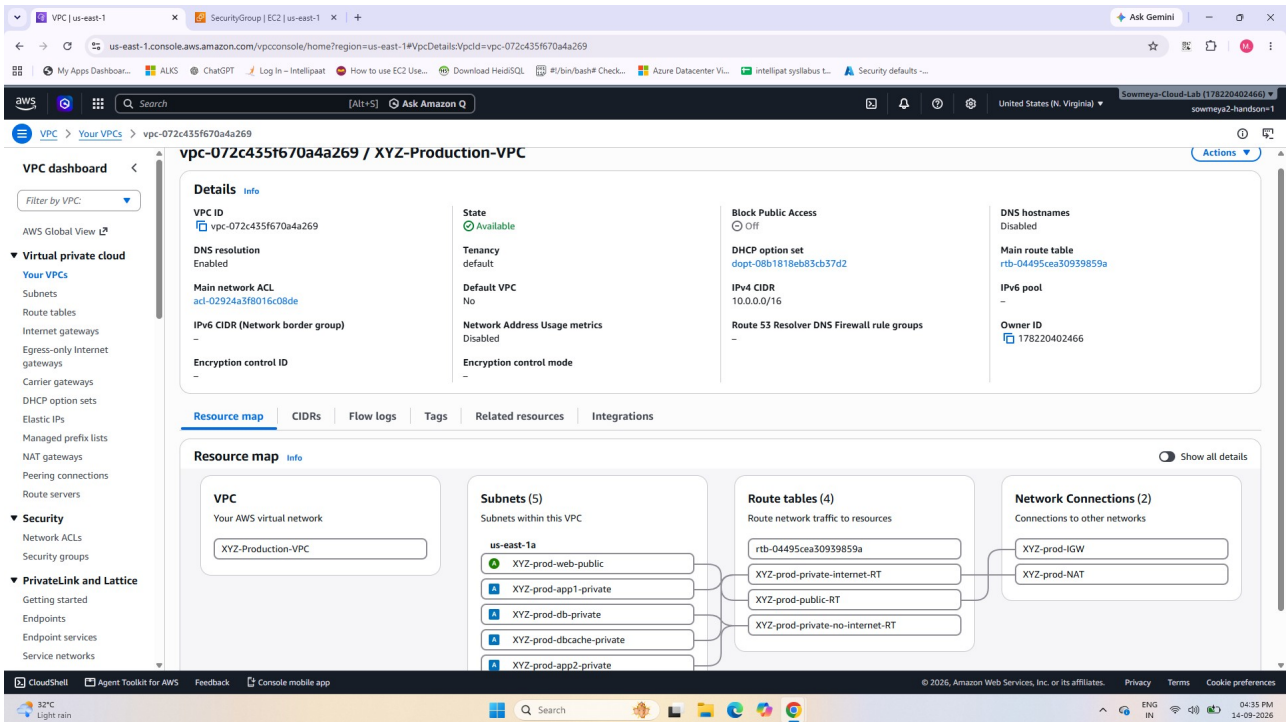


Figure 34: XYZ-Production-VPC resource map – 5 subnets, 4 route tables, IGW and NAT gateway

Development VPC resource map

XYZ-dev-web is connected to XYZ-dev-public-RT and the Internet Gateway; XYZ-dev-db has no path to the internet.

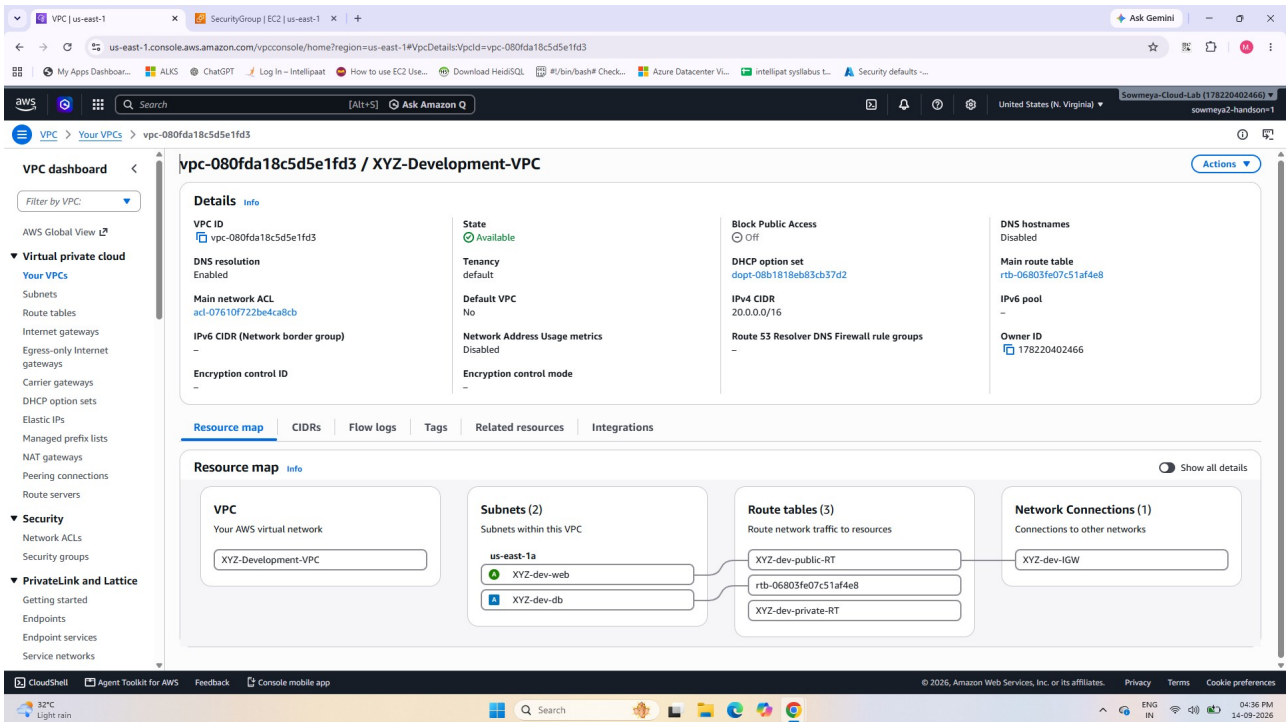


Figure 35: XYZ-Development-VPC resource map – 2 subnets and XYZ-dev-IGW

6. Conclusion

Every requirement in the task maps to the configuration below:

Requirement	How it was met	Evidence
Prod 1 – 4-tier architecture	Separate web, app, cache and db tiers, each with its own route table, NACL and security group	Figures 2, 5, 34
Prod 2 – 5 subnets	XYZ-prod-web-public (public) plus app1, app2, dbcache and db (private)	Figures 2
Prod 3 – Instances in all subnets	XYZ-prod-web, -app1, -app2, -dbcache and -db, each named after its subnet	Figures 14
Prod 4 – dbcache and app1 internet	XYZ-prod-NAT plus XYZ-prod-private-internet-RT associated with app1 and dbcache only	Figures 4, 8, 9
Prod 5 – Security groups and NACLs	One security group per tier and three NACLs (web, app, data)	Figures 12, 13
Dev 1 – 2-tier with instances	XYZ-dev-web and XYZ-dev-db subnets with matching instances	Figures 15, 16, 28
Dev 2 – Only web has internet	Only XYZ-dev-web is on a route table with an Internet Gateway route	Figures 20–23
Dev 3 – Peering connection	XYZ-Prod-Dev-Peering (pcx-00d8cdf7b2ff72646) is Active	Figures 29, 30
Dev 4 – DB-to-DB connection	Peering routes plus MySQL 3306 rules between 10.0.5.0/24 and 20.0.2.0/24	Figures 31–33

With separate production and development VPCs, XYZ Corporation keeps the two environments isolated by default, while the peering connection and tightly scoped security group rules allow only the database tiers to communicate.